

RK-ZYNQ7020-F V1.0 开发板 用户手册

文档变更记录		
版本	时间	描述
V0.1	2024.09.17	初版

目录

一 开发板简介	5
1.1 ZYNQ.....	6
1.2 DDR3	7
1.3 QSPI	7
1.4 EMMC.....	8
1.5 DEBUG.....	8
1.6 SD CARD	9
1.7 USB2.0 HOST/SLAVE	10
1.8 千兆以太网	10
1.9 RS485	11
1.10 CAN	11
1.11 FMC(LPC)	11
1.12 HDMI OUT	11
1.13 MIPI.....	12
1.14 EEPROM	12
1.15 RTC	13
1.16 复位按键.....	13
1.17 LCD	14
1.18 时钟	14
1.19 40PIN IO.....	15
1.20 拨码开关.....	15
1.21 KEY	16
1.22 LED.....	16
1.23 电源.....	16
1.24 结构图.....	18
二 用户快速体验.....	19
1 使用前的准备	19
1.1 串口工具	19
1.2 板卡配件连接	19
2 启动出厂镜像	22
3 使用串口终端测试	25
3.1 ETH 测试.....	25

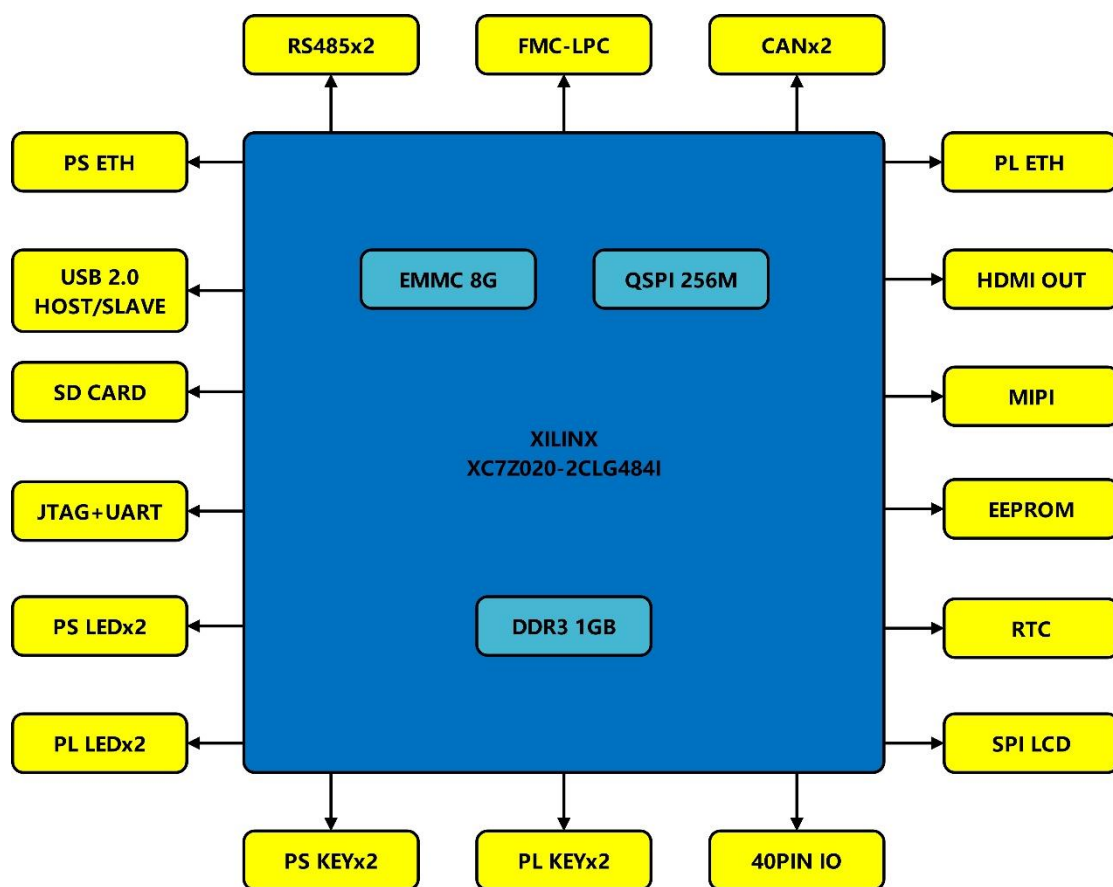
3.2 RS485 测试	28
3.3 CAN 测试	29
3.5 FLASH 测试.....	30
3.6 QSPI 测试	30
3.10 EEPROM 测试.....	31
3.7 LED 测试.....	32
3.8 按键测试	33
3.9 RTC 测试.....	34
三 镜像烧录进 SD 卡	35
四 镜像烧录进 QSPI 卡	36

一 开发板简介

RK-ZYNQ7020-F 开发板采用 Xilinx 公司的 Zynq7000 系列的芯片，型号为 XC7Z020-CLG484-2。ZYNQ7020 芯片可分成处理器系统部分 Processor System (PS) 和可编程逻辑部分 Programmable Logic (PL)。

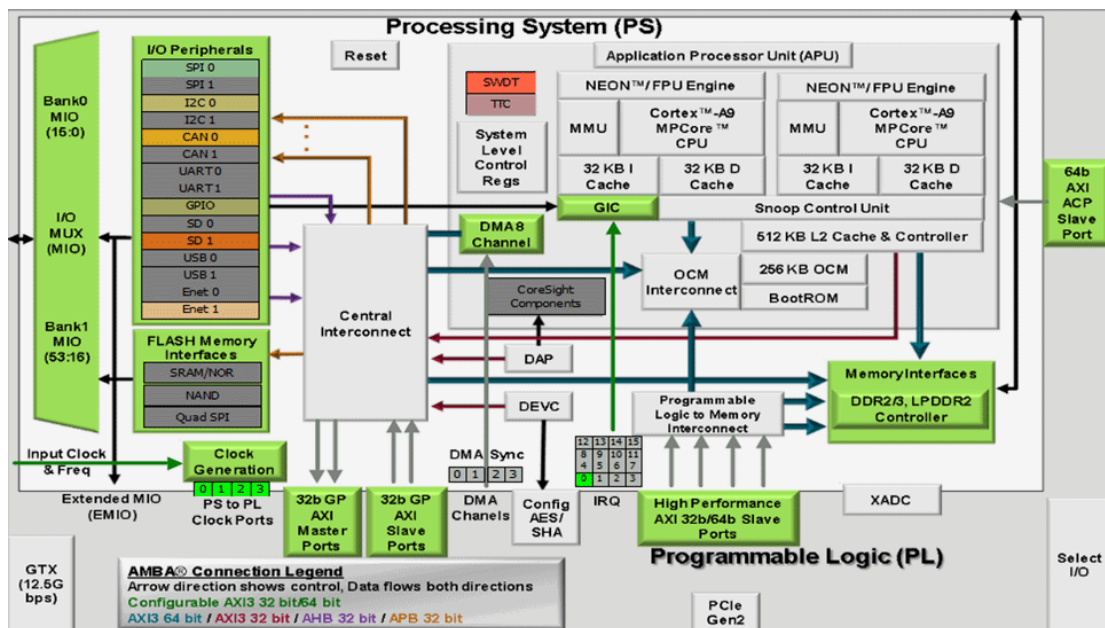
采用整版设计，扩展了丰富的外围接口，包括 DDR3、EMMC、QSPI、板载 JTAG+UART 合一、SD 卡、USB2.0 HOST/SLAVE、PS 千兆以太网、PL 千兆以太网、RS485x2、FMC(LPC)、CANx2、HDMI、MIPI、EEPROM、RTC、SPI LCD、40PIN IO、PS LED KEY、PL LED KEY。

RK-ZYNQ7020-F 开发板结构示意图：



1.1 ZYNQ

开发板使用的是 Xilinx 公司的 Zynq7000 系列的芯片，型号为 XC7Z020-CLG484-2。芯片的 PS 系统集成了两个 ARM Cortex™-A9 处理器，AMBA®互连，内部存储器，外部存储器接口和外设。这些外设主要包括 USB 总线接口，以太网接口，SD/SDIO 接口，I2C 总线接口，CAN 总线接口，UART 接口，GPIO 等。PS 可以独立运行并在上电或复位下启动。ZYNQ7000 芯片的总体框图如下所示：



其中 PS 系统部分的主要参数如下：

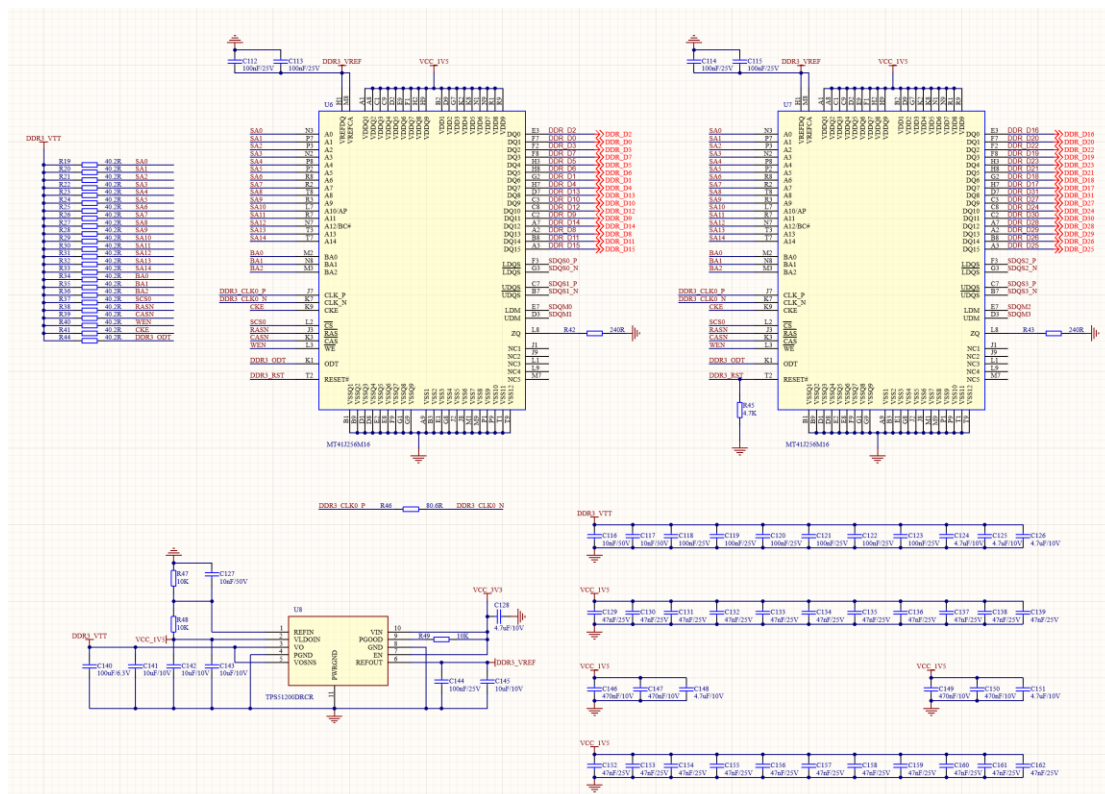
- 基于 ARM 双核 CortexA9 的应用处理器，ARM-v7 架构 高达 1GHz
- 每个 CPU 32KB 1 级指令和数据缓存，512KB 2 级缓存 2 个 CPU 共享
- 片上 boot ROM 和 256KB 片内 RAM
- 外部存储接口，支持 16/32 bit DDR2、DDR3 接口
- 两个千兆网卡支持：发散-聚集 DMA，GMII，RGMII，SGMII 接口
- 两个 USB2.0 OTG 接口，每个最多支持 12 节点
- 两个 CAN2.0B 总线接口
- 两个 SD 卡、SDIO、MMC 兼容控制器
- 2 个 SPI，2 个 UARTs，2 个 I2C 接口
- 4 组 32bit GPIO，54（32+22）作为 PS 系统 IO，64 连接到 PL
- PS 内和 PS 到 PL 的高带宽连接

其中 PL 逻辑部分的主要参数如下：

- 逻辑单元 Logic Cells: 85K;
- 查找表 LUTs: 53,200
- 触发器(flip-flops):106,400
- 乘法器 18x25MACCs: 220;
- Block RAM: 4.9Mb;
- 两个 AD 转换器,可以测量片上电压、温度感应和高达 17 外部差分输入通道，1MBPS

1.2 DDR3

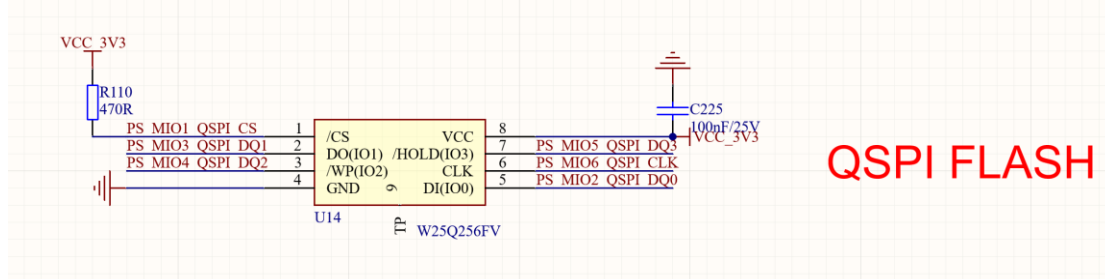
开发板上配有两片 Micron 公司的 DDR3 SDRAM 芯片 (共计 1GB), 型号为 MT41K256M16TW-107:P。DDR3 SDRAM 的总线宽度共为 32bit。DDR3 SDRAM 的最高运行速度可达 533MHz(数据速率 1066Mbps)。该 DDR3 存储系统直接连接到了 ZYNQ 处理系统 (PS) 的 BANK 502 的存储器接口上。



1.3 QSPI

开发板配有一片 256MBit 大小的 Quad-SPI FLASH 芯片, 型号为 W25Q256FVEI, 使用 3.3V CMOS 电压标准。

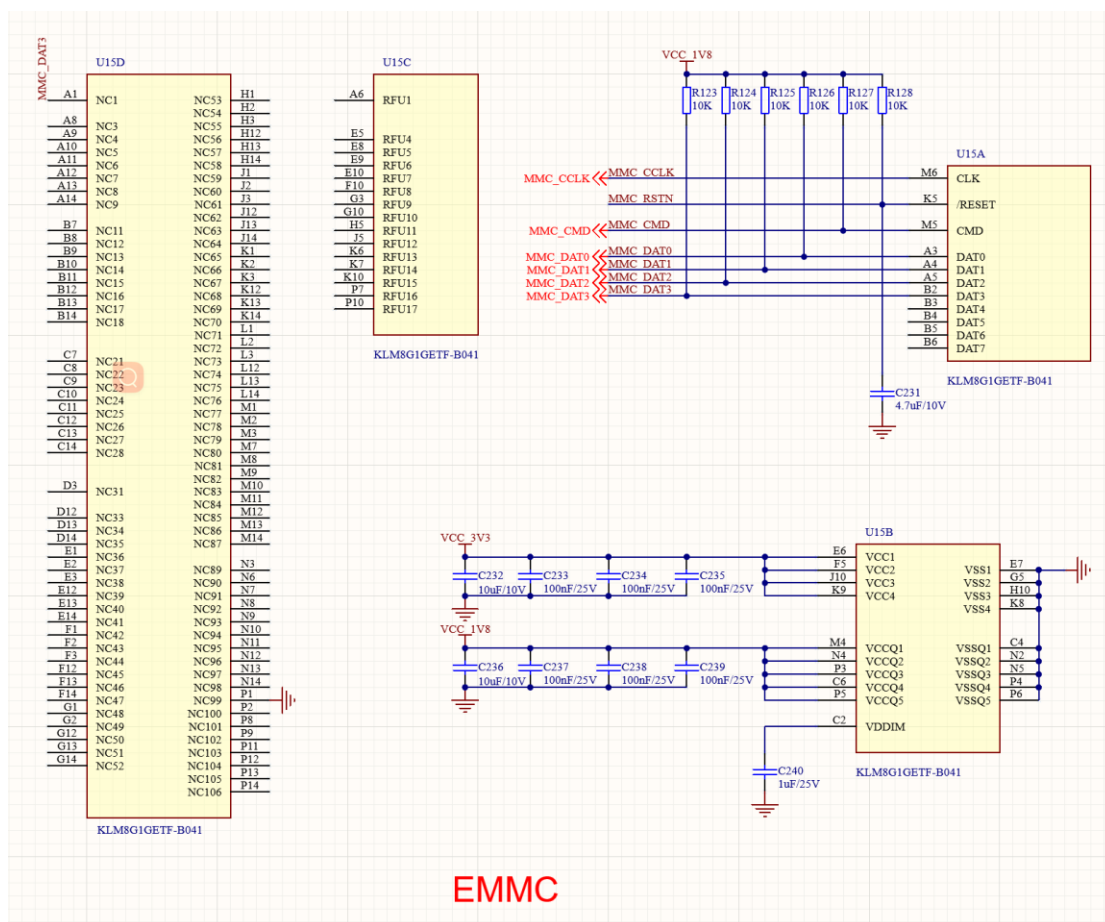
由于 QSPI FLASH 的非易失特性, 在使用中, 它可以作为系统的启动设备来存储系统的启动镜像。这些镜像主要包括 FPGA 的 bit 文件、ARM 的应用程序代码以及其它的用户数据文件。



1.4 EMMC

配有一片大容量的 8GB 大小的 eMMC FLASH 芯片，型号为 KLM8G1GETF-B041，它支持 JEDEC e-MMC V5.0 标准的 HS-MMC 接口，电平支持 1.8V 或者 3.3V。eMMC FLASH 和 ZYNQ 连接的数据宽度为 4bit。

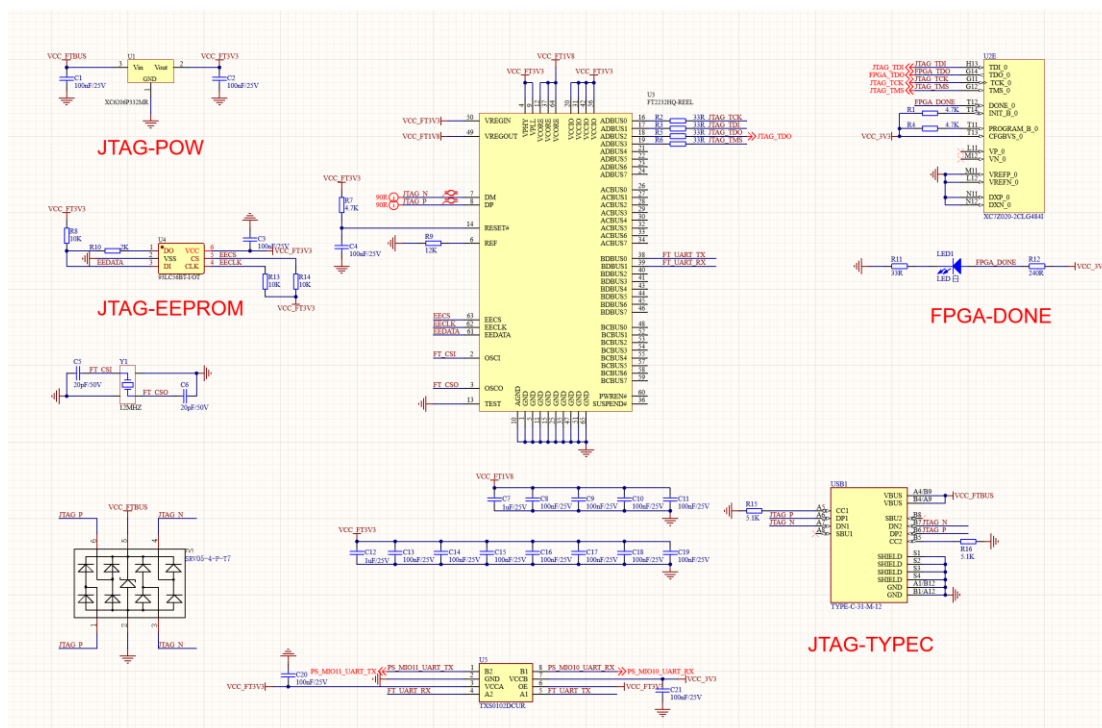
由于 eMMC FLASH 的大容量和非易失特性，在 ZYNQ 系统使用中，它可以作为系统大容量的存储设备，比如存储 ARM 的应用程序、系统文件以及其它的用户数据文件。



EMMC

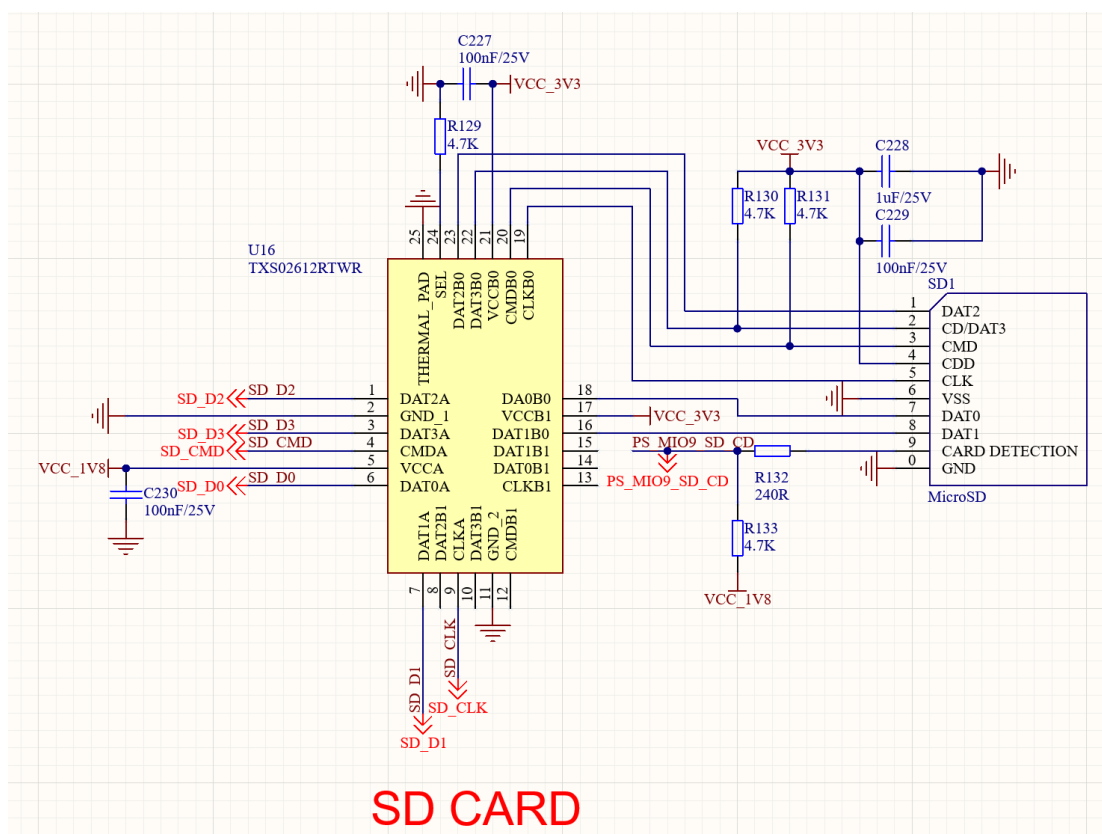
1.5 DEBUG

开发板使用了一个 FT2232HQ 芯片，转换出一个 UART、一个 JTAG 电路。FT2232HQ 是一颗双通道的下载器芯片。可通过一个 USB 线，在电脑上识别到多个设备，其中包括我们的串口设备和 JTAG 转换设备。下载器最高支持 30M，可完成赛灵思 FPGA 及 ARM 的在线调试以及 ARM 的代码断点等基本调试功能。



1.6 SD CARD

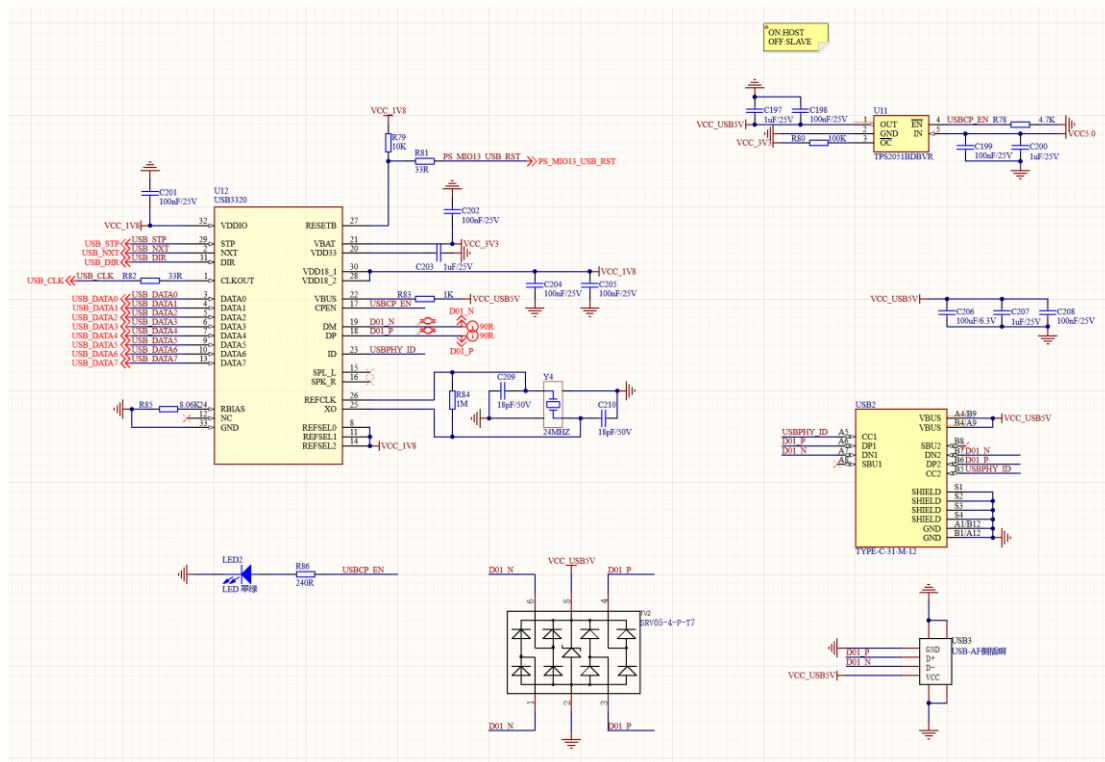
开发板包含了一个 Micro 型的 SD 卡接口，以提供用户访问 SD 卡存储器，用于存储 BOOT 程序，Linux 操作系统内核，文件系统以及其它的用户数据文件。



1.7 USB2.0 HOST/SLAVE

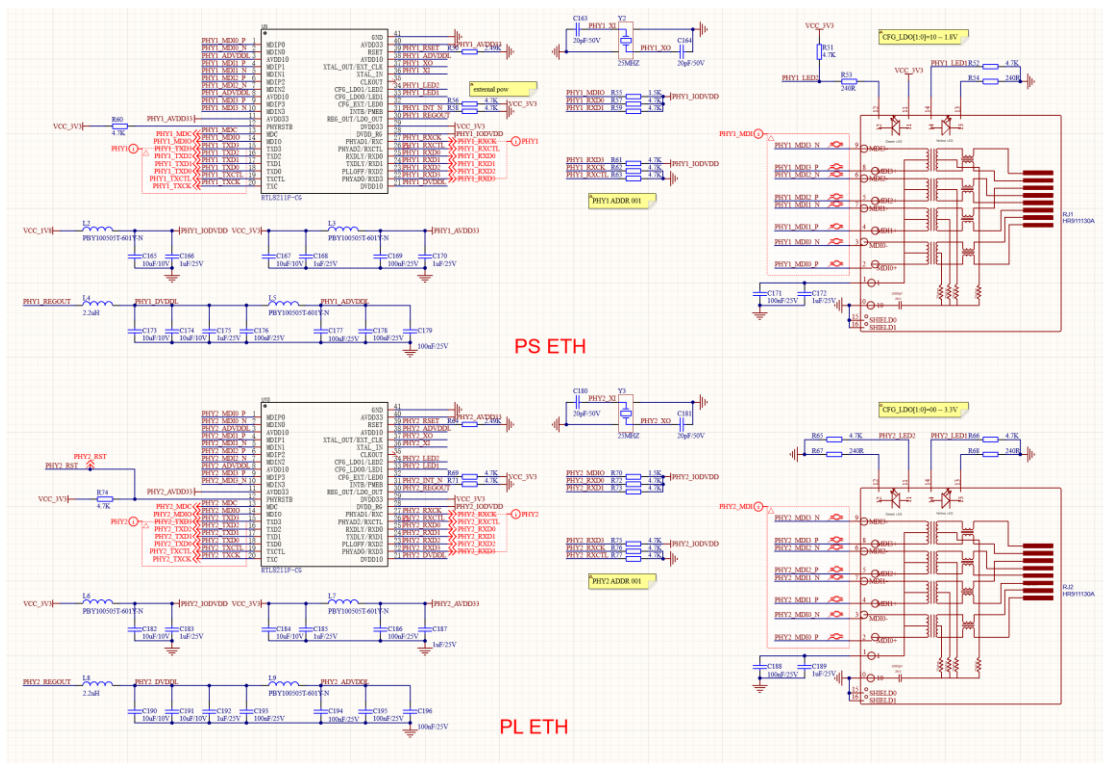
开发板有 1 个 USB2.0 HOST/SLAVE 接口, USB2.0 收发器采用的是一个 1.8V 的, 高速的支持 ULPI 标准接口的 **USB3320C-EZK** 芯片,ZYNQ 的 USB 总线接口和 USB3320C-EZK 收发器相连接, 实现高速的 USB2.0 Host 模式的数据通信。USB3320C 的 USB 的数据和控制信号连接到 ZYNQ 芯片 PS 端的 BANK501 的 IO 口上。24MHz 的晶振为为 USB3320C 芯片提供系统时钟。

USB 接口为扁型 USB 接口(USB Type A 和 TYPEC), 方便用户同时连接不同的 USB Slave 外设(比如 USB 鼠标和 USB 键盘), 也可使用 TYPEC 连接主机。另外开发板为 USB HOST 接口提供了+5V 的电源。



1.8 千兆以太网

开发板有 2 路千兆以太网接口, 1 路连接到 PS 端, 另 1 路连接到 PL 端。PHY 芯片采用 REALTEK 公司的 **RTL8211F-CG** 以太网 PHY 芯片为用户提供网络通信服务。RTL8211F-CG 芯片支持 10/100/1000 Mbps 网络传输速率, 通过 RGMII 接口跟 ZYNQ7020 系统的 MAC 层进行数据通信。



1.9 RS485

开发板有 2 路 RS485 接口，连接在 BANK33 上。RS485 收发芯片选用 **SP3485EN-L/TR** 为用户提供 RS485 通信服务。

1.10 CAN

开发板上有 2 路 CAN 接口，连接在 BANK33 上。CAN 收发芯片选用 **SN65HVD230DR** 为用户提供 CAN 通信服务。

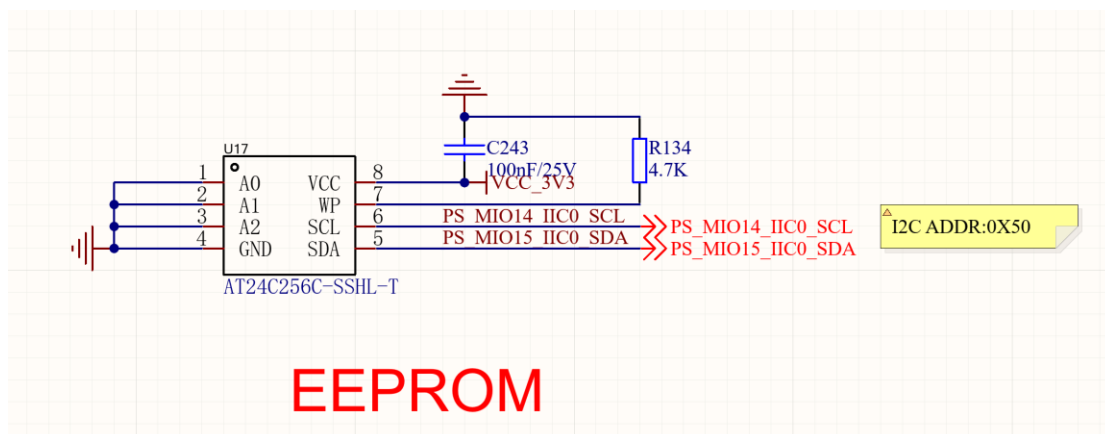
1.11 FMC(LPC)

开发板上配备了一个标准的 FMC LPC 的扩展口，可以连接各种 FMC 模块。FMC 扩展口包含 36 对差分 IO 信号（全部等长）。

FMC 扩展口的 36 对差分信号连接到 ZYNQ7020 的 BANK34, 35 的 IO 上，电平标准默认为 2.5V（可更改），差分信号支持 LVDS 数据通信，**且完全兼容 ZEDBOARD 的 FMC**。

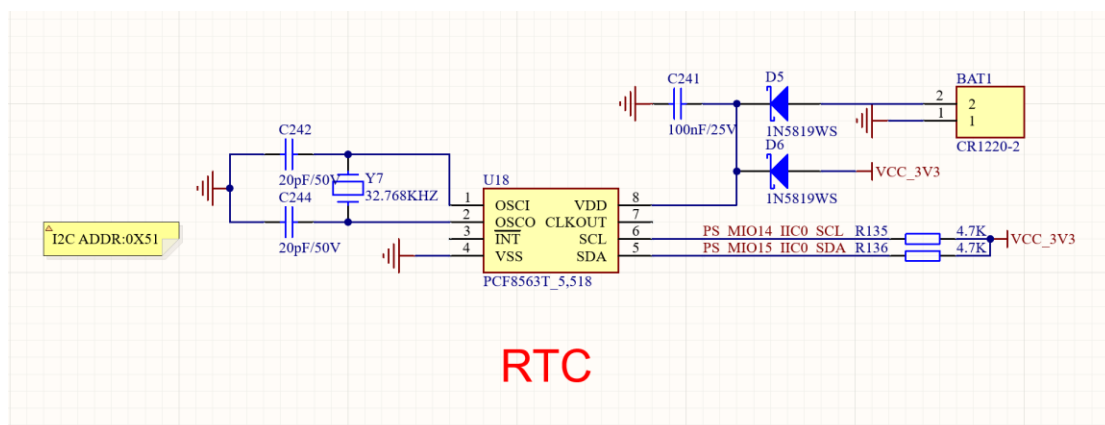
1.12 HDMI OUT

开发板上通过 ZYNQ 的差分 IO 直接连接到 HDMI 接口的差分信号和时钟，在 ZYNQ 内部实现 HDMI 信号的差分转并行再进行编解码，实现 DVI 数字视频输出，最高支持



1.15 RTC

开发板搭载了一片实时时钟 RTC 芯片，型号 PCF8563T_5,518。外部需要接一个 32.768KHz 的无源时钟，提供精确的时钟源给时钟芯片，这样才能让 RTC 可以准确的提供时钟信息给产品。同时为了产品掉电以后，实时时钟还可以正常运行，一般需要另外配一个电池给时钟芯片供电，将纽扣电池（型号 CR1220，电压为 3V）放入以后，当系统掉电后，纽扣电池还可以给 RTC 芯片供电，提供持续不断的时间信息。



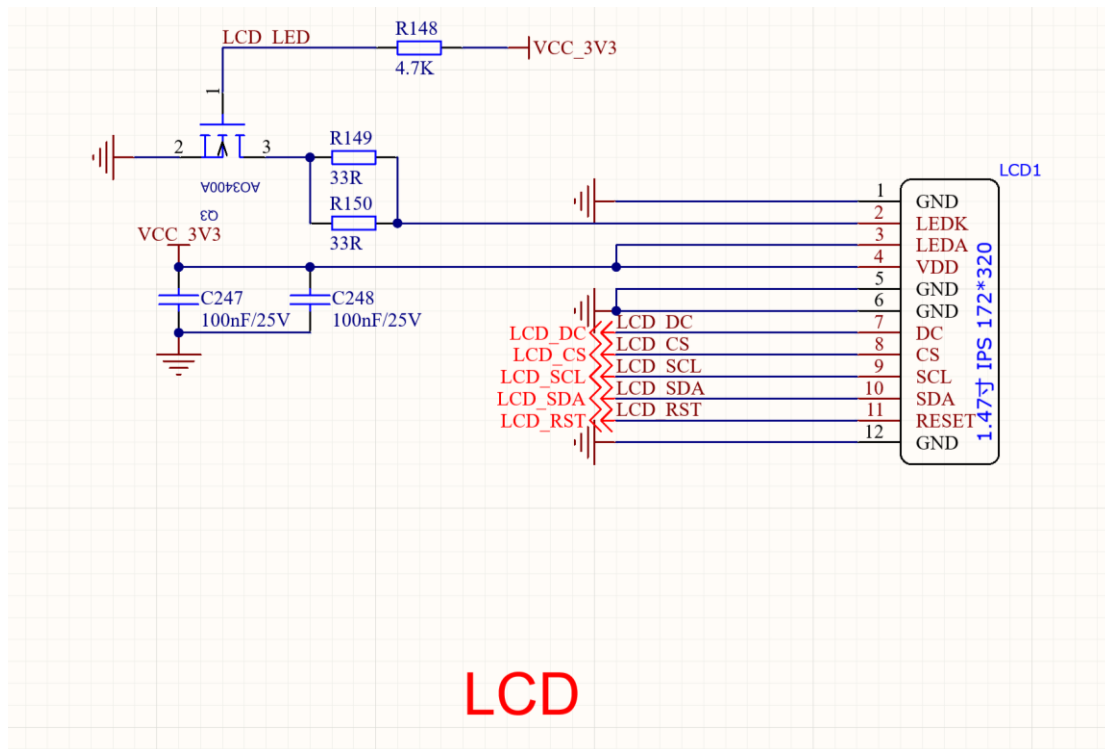
1.16 复位按键

开发板配备一个复位按键，用于芯片复位。



1.17 LCD

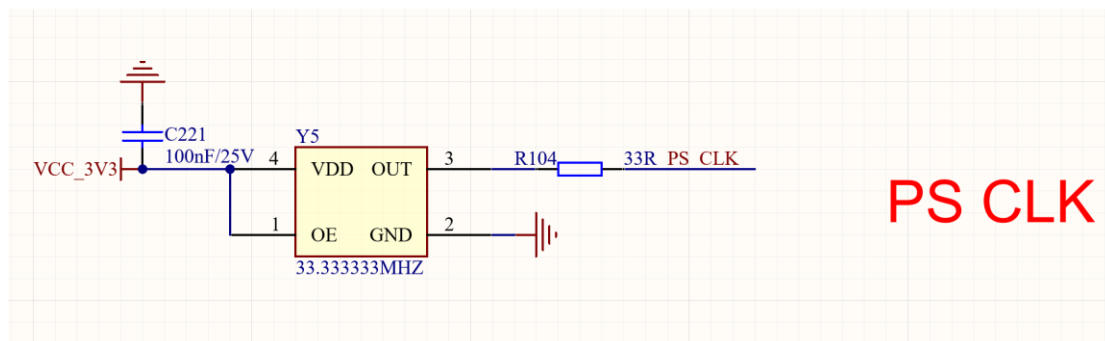
开发板带一个 1.47 寸的 SPI LCD 屏，接在 PL 测（PS 也可使用），分辨率为 320X172。



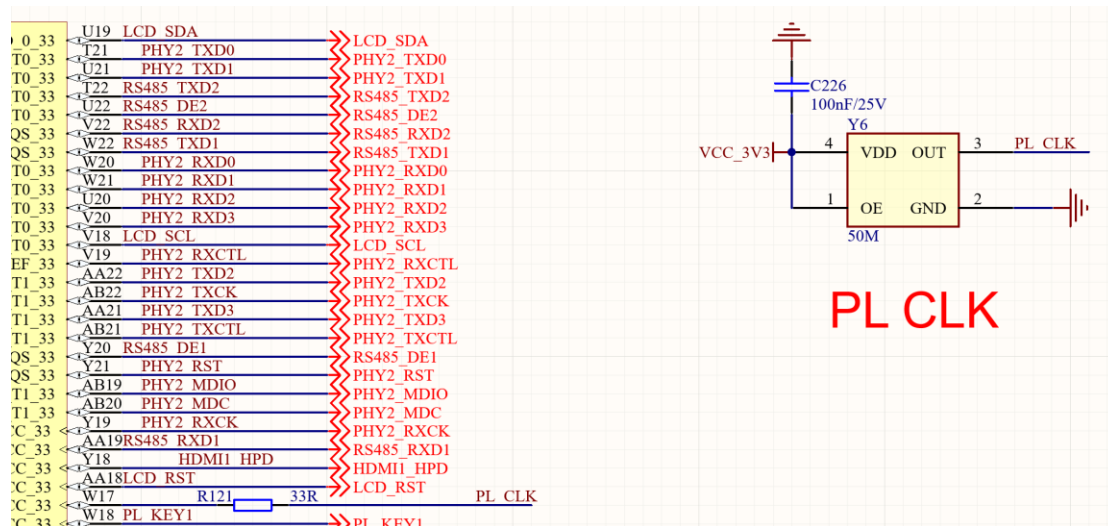
1.18 时钟

开发板上分别为 PS 系统, PL 逻辑部分提供了参考时钟，使 PS 系统和 PL 逻辑可以单独工作。

- (1) 一颗 33.333MHZ 的有源晶振，作为 ZYNQ7020 的 PS 系统的时钟输入；



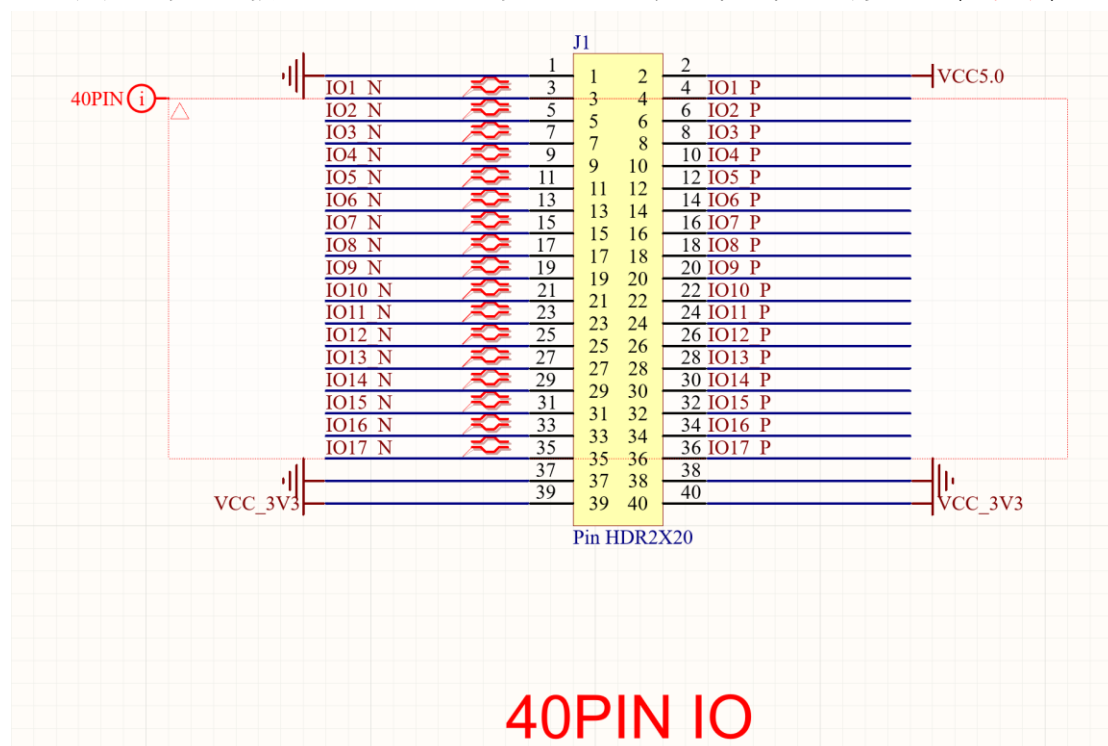
- (2) 一颗 50MHZ 的有源晶振，作为 ZYNQ7020 的 PL 系统的时钟输入；



1.19 40PIN IO

开发板留有一个 2.54mm 间距的 40 针的扩展口（兼容黑金），用于连接黑金的各个模块或者用户自己设计的外围电路，扩展口有 40 个信号，其中，5V 电源 1 路，3.3V 电源 2 路，GND 3 路，IO 34 路（17 对差分走线，且全部等长）。

扩展口的 IO 连接至 FPGA 芯片 PL 的 BANK13 上，电平标准默认为 3.3V（可更改）。



1.20 拨码开关

开发板上的两位拨码开关可用于启动模式的选择（仅下图红框内的启动模式可选），拨码开关 ON 侧为 0。

Table 6-2: Boot_Mode Strapping MIO Pins

	vmode[1]	vmode[0]	BOOT_MODE [4]	BOOT_MODE [0]	BOOT_MODE [2]	BOOT_MODE [1]	BOOT_MODE [3]
	MIO[8]	MIO[7]	MIO[6]	MIO[5]	MIO[4]	MIO[3]	MIO[2]
Cascaded JTAG							0
Independent JTAG							1
Boot Devices							
JTAG				0	0	0	0
NOR				0	0	1	
NAND				0	1	0	
Reserved				0	1	1	
Quad-SPI				1	0	0	
Reserved				1	0	1	
SD				1	1	0	
Reserved				1	1	1	

1.21 KEY

开发板上包含 2 个 PS 侧按键、2 个 PL 侧按键。

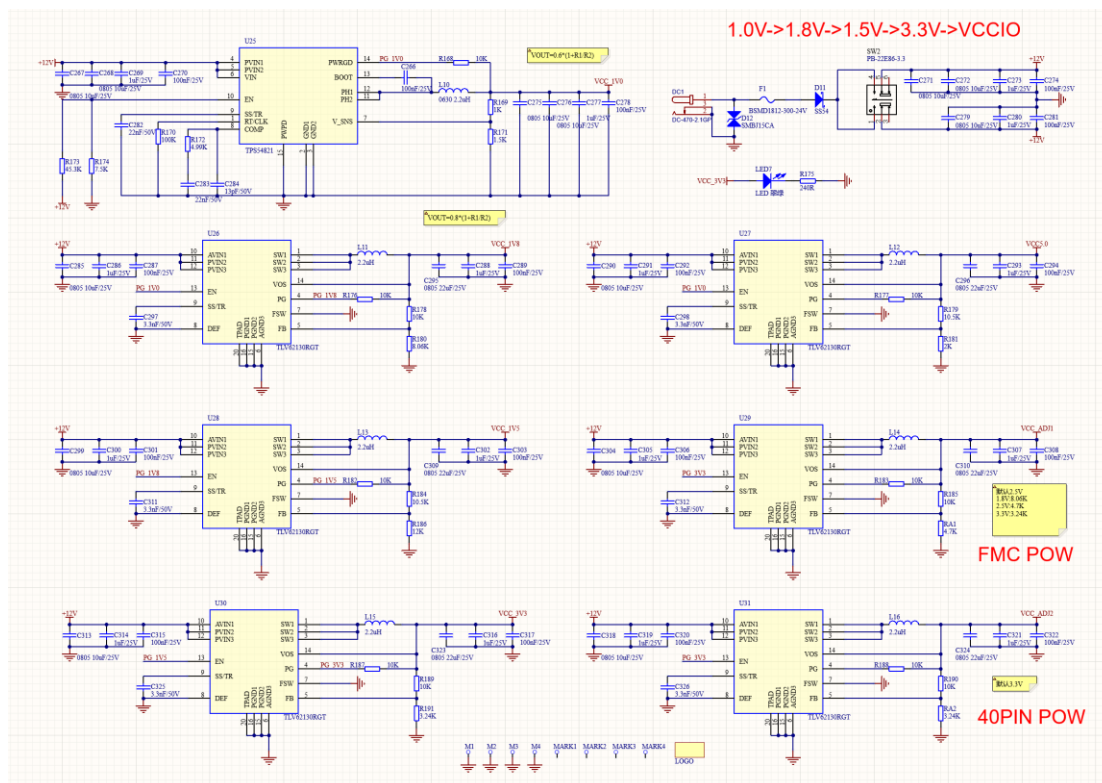
1.22 LED

开发板上包含 2 个 PS 侧 LED、2 个 PL 侧 LED 和一个电源指示灯。

1.23 电源

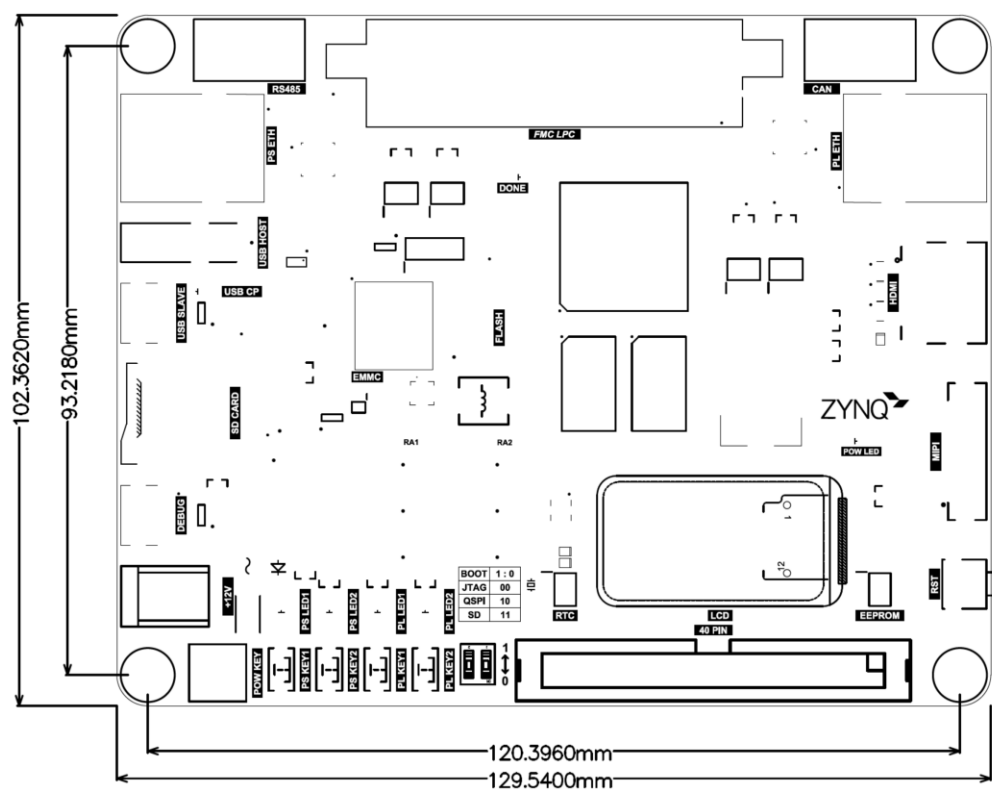
开发板的输入电源为 12V(输入范围 10-14V)，配有一个按键开关来控制电源开关。输入带有过流、防反接及 TVS 过压保护功能。

其中 FMC 和 40PIN IO 电压可更改为 1.8V/2.5V/3.3V，只需修改 RA 标识的电阻即可。



1.24 结构图

开发板的尺寸为：129.54x102.36x35.3（单位：mm）



二 用户快速体验

1 使用前的准备

出厂前开发板 QSPI 和 EMMC 已烧写 LINUX 系统文件。

1.1 串口工具

MobaXterm 下载链接：

[MobaXterm Xserver with SSH, telnet, RDP, VNC and X11 - Download \(mobatek.net\)](https://mobatek.net/mobaxterm/download/)

FT2232HQ 驱动在安装 vivado 时已经安装上了。

以上软件安装较为简单，直接点安装和下一步。

必须物品：

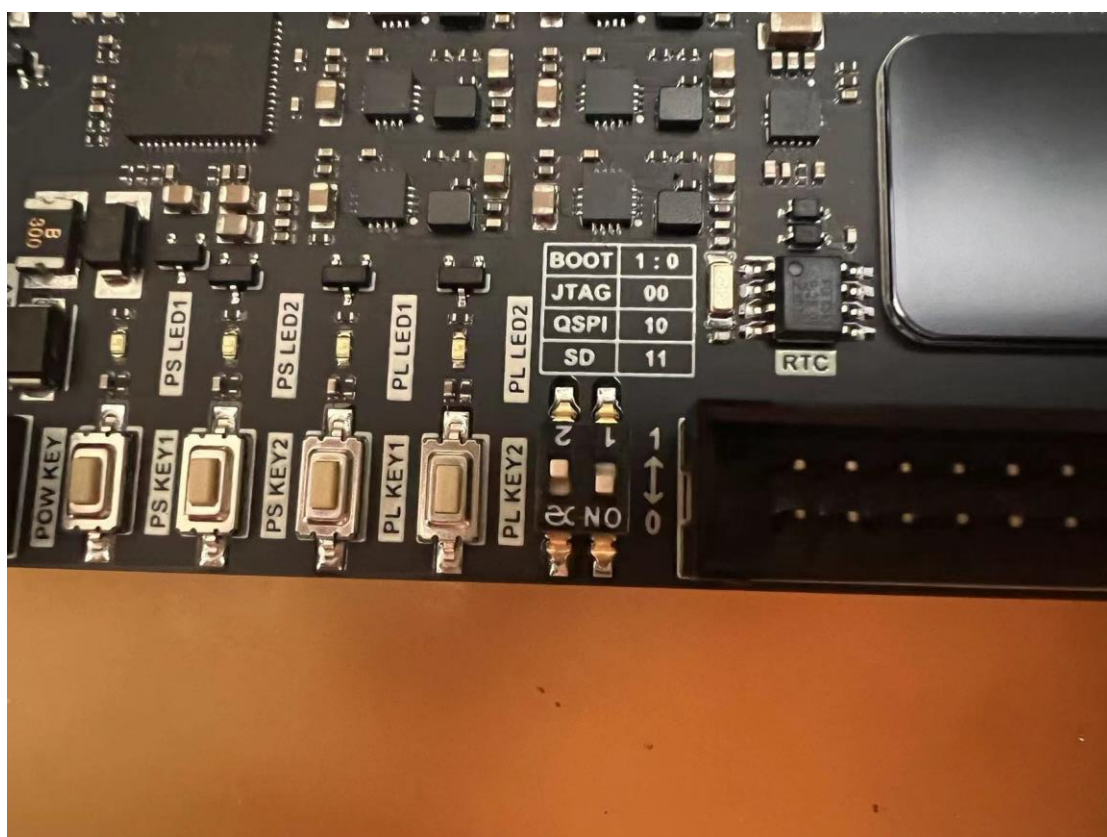
序号	名称	数量
1	测试版卡	1
2	TYPE-C 线束	1
3	HDMI 接口显示器 HDMI 线束	1
4	电脑	1
5	鼠标或键盘（至少一个）	1
6	12V3A 电源 (5.5mm×2.1 或 2.5mm)	1

选配物品

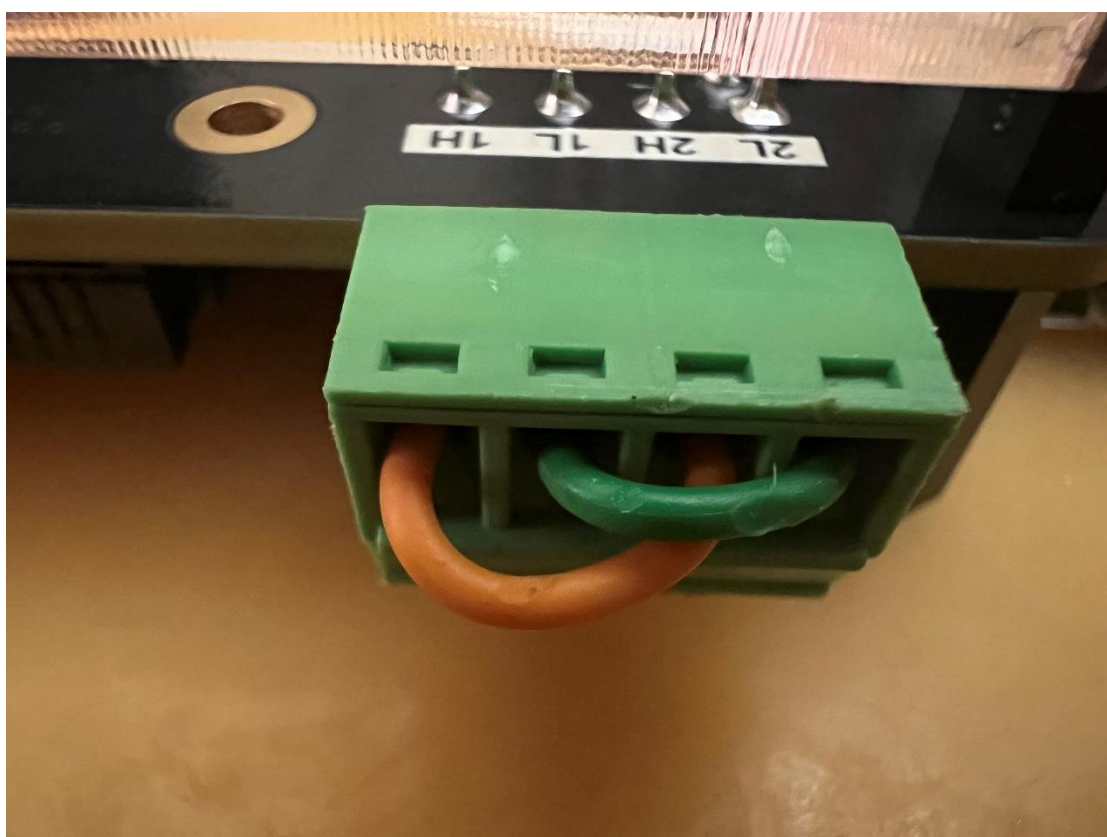
序号	名称	数量
1	RJ45 网线	2
2	杜邦线	4
3	AN5641 摄像头	1
4	SD 卡（2.0、8G-32G）	1

1.2 板卡配件连接

1. 拨码开关配置为 QSPI 启动（拨码开关 1：0→10，上下，ON 侧为 0）

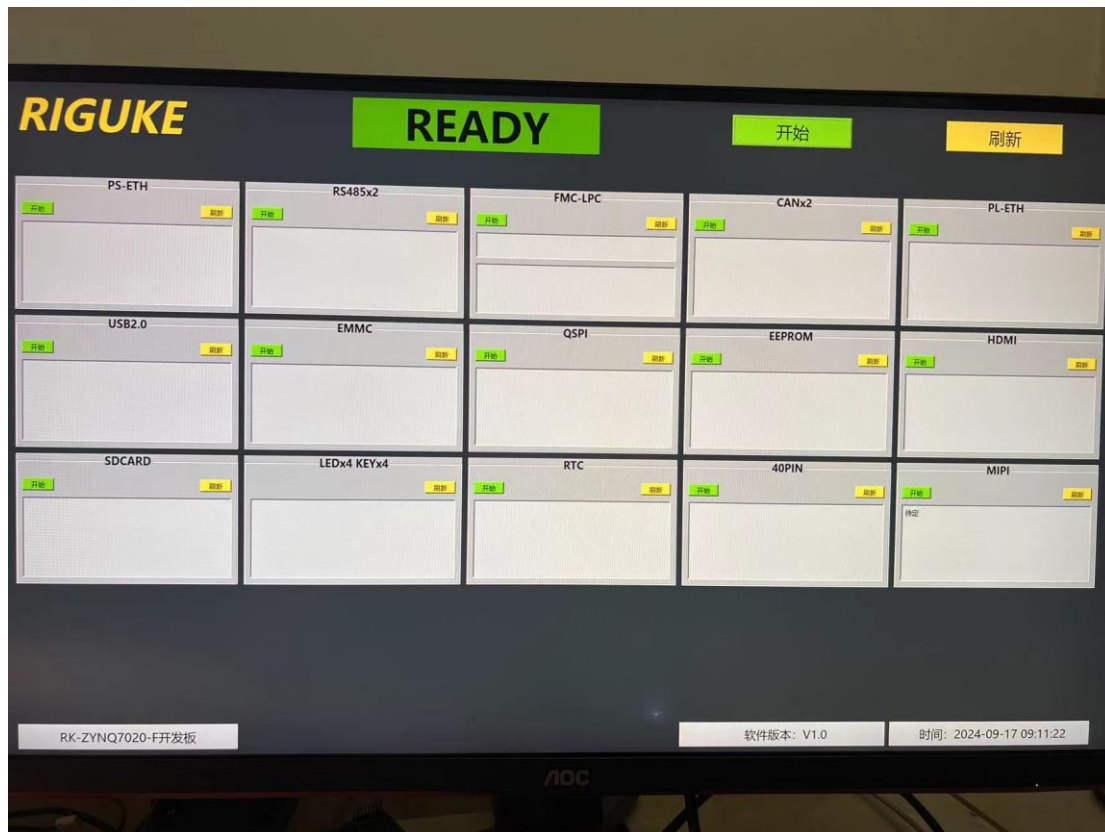


2. CAN 使用杜邦线或配送的公头连接器如下短接，RS485 同理

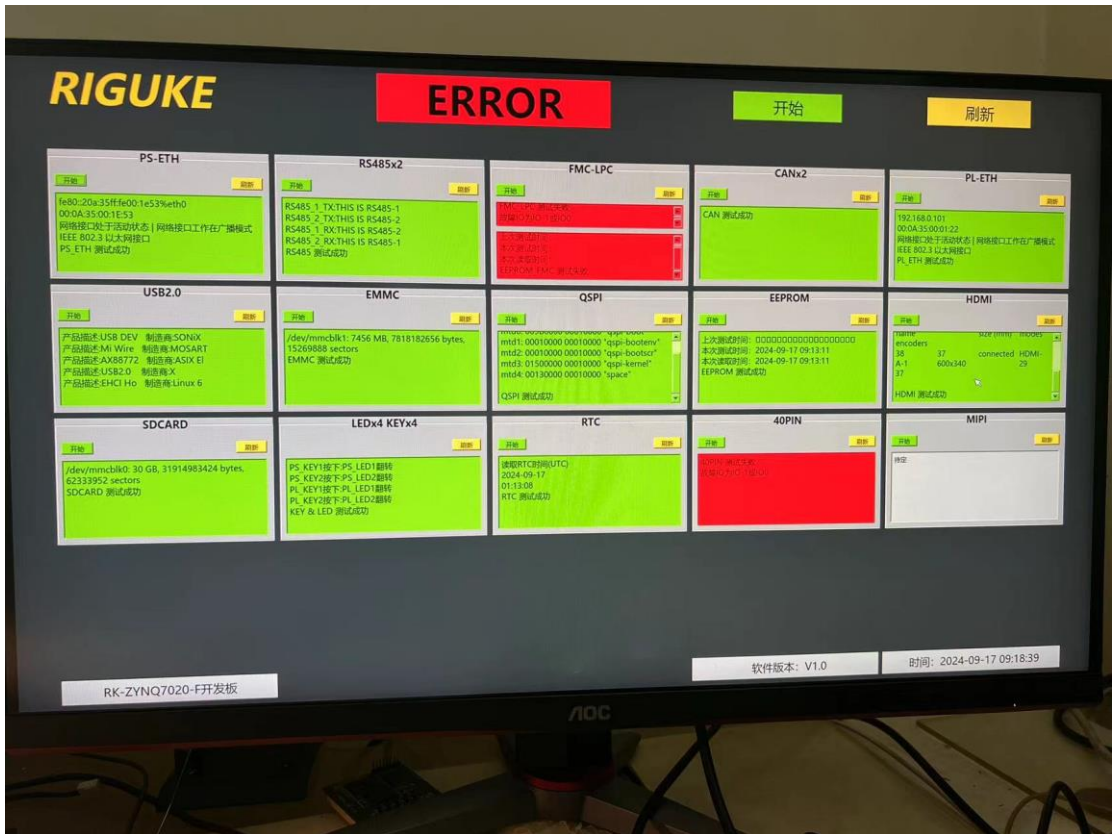


2 启动出厂镜像

1. 电脑打开 MobaXterm 串口工具，打开串口
2. 按下电源开关，启动出厂镜像
3. 串口工具出现命令行，且 HDMI 显示测试程序，则启动完成



4. 按下连接测试版卡的键盘回车键或鼠标点击开始，则开始测试
各项通过条件
 - 1) PS-ETH 协商成功，若连接 DHCP 主机，则会显示 IP 地址，同理 PL-ETH
 - 2) CAN×2 做回环测试，短接即可通过测试，同理 RS485×2
 - 3) USB2.0 测试结果显示 USB 设备信息
 - 4) EMMC 测试结果显示其容量，同理 QSPI、SDCARD
 - 5) EEPROM 先读出上次测试时间，并写入本次测试时间并再读出
 - 6) HDMI 测试结果为显示器的 ID
 - 7) LED×4 KEY×4，按下按键，对应的 LED 灯翻转，四个按键全部按过一次则通过
 - 8) RTC 测试结果为读出内部时间（UTC 时间，慢八小时）
 - 9) 40PIN & FMC-HPC 为工装测试，不用在意



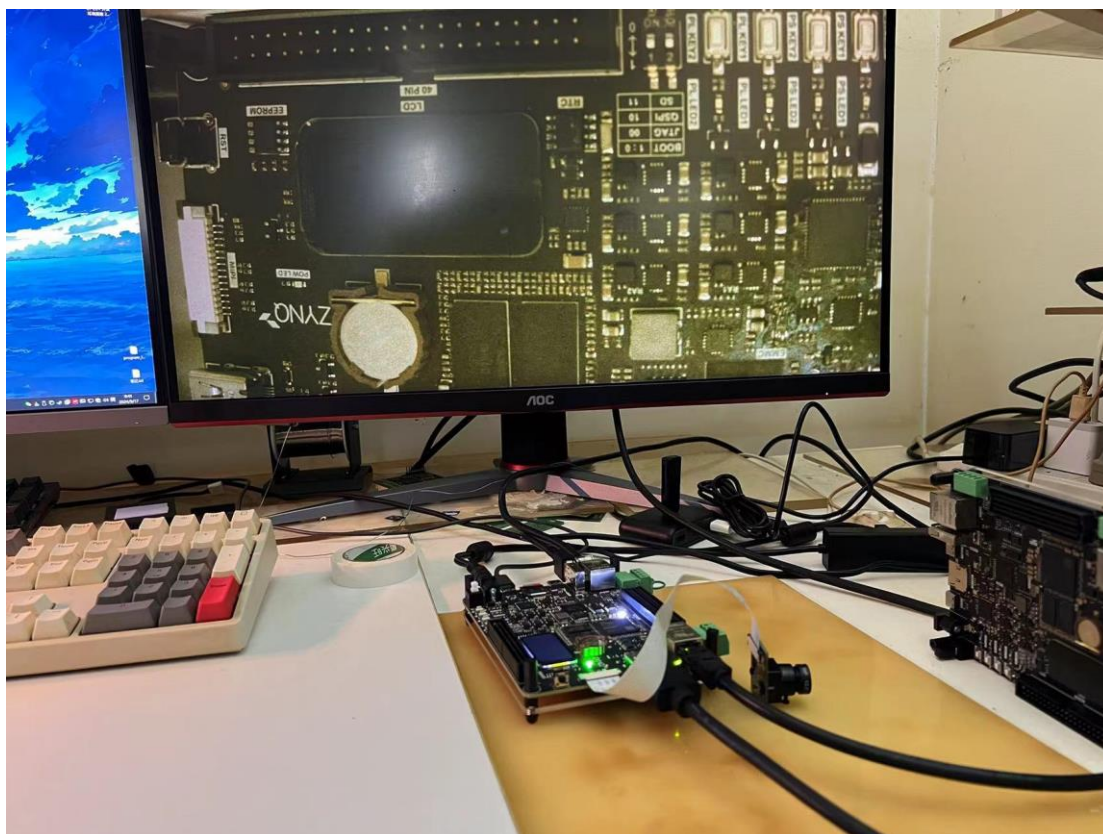
10) mipi: 暂未添加到出厂镜像内

请使用以下固件复制到 FAT32 格式的 SD 卡中测试，过程不再赘述

6.RK-ZYNQ7020-F开发板网盘资料 > 5_出厂相关 > vdma_mipi				在 vdma_mipi 中搜索
名称	修改日期	类型	大小	
BOOT.BIN	2024/8/27 19:59	BIN 文件	4,229 KB	

打印日志如下:

```
Gamma correction LUT initialized
Demosaic module initialized
Got ov5640 id, 56 40
Frame counter is 32369
Frame counter is 64769
crc value is 0
```



3 使用串口终端测试

使用串口终端测试前，请先关闭测试程序，避免端口占用的情况：

```
ps |grep qt
kill 569
```

```
root@image_7020:~# ps |grep qt
569 root      129m S    ./opt/qt_7020/bin/qt_7020
693 root      2384 S    grep qt
root@image_7020:~# kill 569
root@image_7020:~#
```

3.1 ETH 测试

开发板在 PS 侧和 PL 侧各有一个网口。

测试时网口需连接至带有 DHCP 服务的主机或路由器。

查看系统中的所有网络设备，eth0 为 PS ETH，eth1 为 PL ETH：

```
ifconfig -a
```

```
root@image_7020:~# ifconfig -a
can0: flags=193<UP,RUNNING,NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 1 bytes 5 (5.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1 bytes 5 (5.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 44

can1: flags=193<UP,RUNNING,NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 1 bytes 5 (5.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1 bytes 5 (5.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 45

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.100 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::20a:35ff:fe00:1e53 prefixlen 64 scopeid 0x20<link>
    ether 00:0a:35:00:1e:53 txqueuelen 1000 (Ethernet)
    RX packets 51 bytes 3903 (3.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 31 bytes 4970 (4.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 46 base 0xb000

eth1: flags=67<UP,BROADCAST,RUNNING> mtu 1500
    inet 192.168.0.101 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::20a:35ff:fe00:122 prefixlen 64 scopeid 0x20<link>
    ether 00:0a:35:00:01:22 txqueuelen 1000 (Ethernet)
    RX packets 105 bytes 9077 (8.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 71 bytes 7988 (7.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 194 bytes 14156 (13.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 194 bytes 14156 (13.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sit0: flags=128<NOARP> mtu 1480
    unspec 00-00-00-00-31-00-00-9B-00-00-00-00-00-00-00-00 txqueuelen 1000 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@image_7020:~#
```

两个 ETH 默认已开启，并连接至具有 DHCP 服务端功能的主机，开机后也可使用以下命令通过 DHCP 使相应网口获取地址：

```
udhcpc -i eth1
```

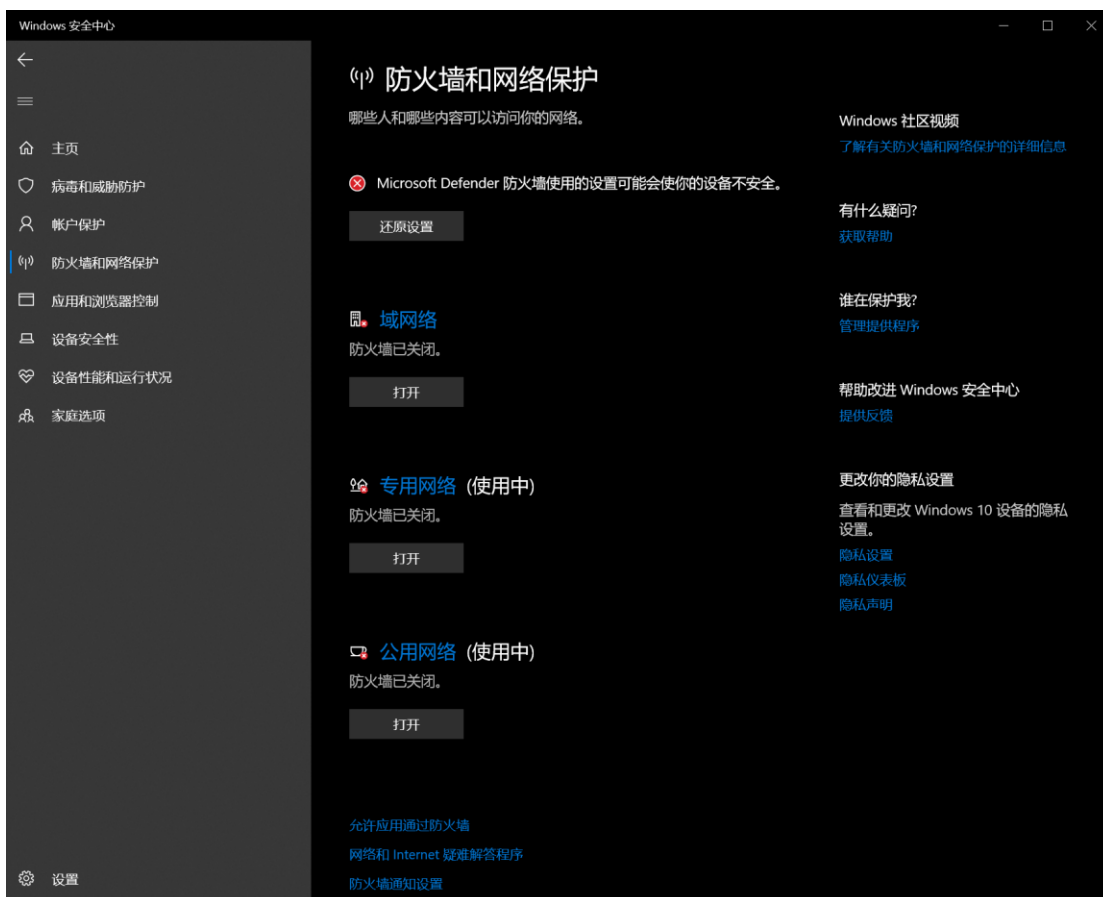
```

root@image_7020:~# udhcpd -i eth1
udhcpd: started, v1.35.0
udhcpd: broadcasting discover
udhcpd: broadcasting select for 192.168.0.101, server 192.168.0.1
udhcpd: lease of 192.168.0.101 obtained from 192.168.0.1, lease time 7200
/etc/udhcpd.d/50default: Adding DNS 192.168.0.1
root@image_7020:~#

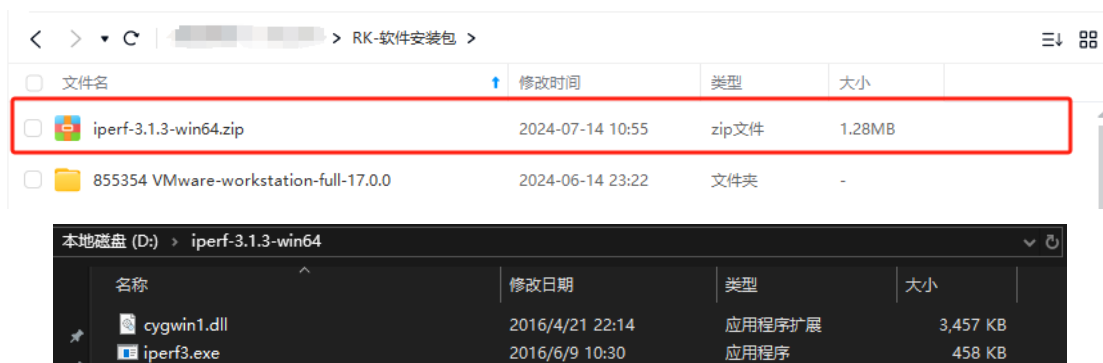
```

下面将使用 iperf3 测试网口速率

(1) 将 windows 防火墙关闭



(2) 下载网盘中的 iperf3 工具，并解压至磁盘中



(3) 测电脑下发到开发板的速率:

开发板作为服务器端，电脑作为客户端进行测试。在开发板串口终端中执行以下命令：

iperf3 -s

```

root@image_7020:~# iperf3 -s
-----
Server listening on 5201 (test #1)
-----

```

在电脑的 cmd 命令提示符中执行以下命令：

```
cd /d D:\iperf-3.1.3-win64
iperf -c 192.168.1.100
```

```
D:\iperf-3.1.3-win64>iperf3 -c 192.168.1.100
Connecting to host 192.168.1.100, port 5201
[ 4] local 192.168.1.111 port 4808 connected to 192.168.1.100 port 5201
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.00-1.00 sec  96.2 MBytes 807 Mbits/sec
[ 4] 1.00-2.00 sec  96.0 MBytes 805 Mbits/sec
[ 4] 2.00-3.00 sec  95.9 MBytes 804 Mbits/sec
[ 4] 3.00-4.00 sec  96.2 MBytes 807 Mbits/sec
[ 4] 4.00-5.00 sec  96.1 MBytes 807 Mbits/sec
[ 4] 5.00-6.00 sec  96.4 MBytes 808 Mbits/sec
[ 4] 6.00-7.00 sec  96.4 MBytes 808 Mbits/sec
[ 4] 7.00-8.00 sec  96.2 MBytes 808 Mbits/sec
[ 4] 8.00-9.00 sec  96.1 MBytes 807 Mbits/sec
[ 4] 9.00-10.00 sec 96.4 MBytes 809 Mbits/sec
--
[ ID] Interval      Transfer    Bandwidth          sender
[ 4] 0.00-10.00 sec 962 MBytes 807 Mbits/sec
[ 4] 0.00-10.00 sec 962 MBytes 807 Mbits/sec          receiver
iperf Done.
D:\iperf-3.1.3-win64>
```

执行上面的命令共打印出了 10 次带宽测试报告，每一秒钟报告一次带宽，耗费时间为 10 秒钟；平均带宽表示 10 次带宽报告所得出的平均值，包括上传带宽（sender）和下载带宽（receiver）。

注：此处的带宽值跟电脑有关，不同的电脑值有差异，一般大于 500Mb/s 即可。

由此可以知道开发板上的 PS 网口是千兆以太网接口 (1000Mbits > 带宽 > 100Mbits)。测试完成后，在串口终端中按“CTRL+C”结束 iperf3。

(4) 测开发板上传到电脑的速率：

电脑作为服务器端，开发板作为客户端进行测试。在电脑的 cmd 命令提示符中执行以下命令：

```
iperf3 -s
```

```
D:\iperf-3.1.3-win64>iperf3 -s
Server listening on 5201
```

然后在开发板的串口终端执行以下命令：

```
iperf3 -c 192.168.1.111
```

```
root@image_7020:~# iperf3 -c 192.168.1.111
Connecting to host 192.168.1.111, port 5201
[ 5] local 192.168.1.100 port 46916 connected to 192.168.1.111 port 5201
[ ID] Interval      Transfer    Bitrate    Retr  Cwnd
[ 5] 0.00-1.01 sec  82.5 MBytes 689 Mbits/sec  0    433 KBytes
[ 5] 1.01-2.00 sec  83.8 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 2.00-3.00 sec  83.8 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 3.00-4.01 sec  85.0 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 4.01-5.01 sec  83.8 MBytes 703 Mbits/sec  0    433 KBytes
[ 5] 5.01-6.01 sec  83.8 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 6.01-7.01 sec  83.8 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 7.01-8.00 sec  83.8 MBytes 705 Mbits/sec  0    433 KBytes
[ 5] 8.00-9.00 sec  83.8 MBytes 704 Mbits/sec  0    433 KBytes
[ 5] 9.00-10.01 sec 85.0 MBytes 704 Mbits/sec  0    433 KBytes
--
[ ID] Interval      Transfer    Bitrate    Retr
[ 5] 0.00-10.01 sec  839 MBytes 703 Mbits/sec  0
[ 5] 0.00-10.01 sec  839 MBytes 703 Mbits/sec
iperf Done.
root@image_7020:~#
```

同理 PL ETH 的测试，不过多赘述：

```
D:\iperf-3.1.3-win64>iperf3 -c 192.168.0.101
Connecting to host 192.168.0.101, port 5201
[ 4] local 192.168.0.102 port 5988 connected to 192.168.0.101 port 5201
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-1.00    sec 60.9 MBytes 511 Mbits/sec
[ 4] 1.00-2.00    sec 67.6 MBytes 567 Mbits/sec
[ 4] 2.00-3.00    sec 69.8 MBytes 585 Mbits/sec
[ 4] 3.00-4.00    sec 65.9 MBytes 553 Mbits/sec
[ 4] 4.00-5.00    sec 66.1 MBytes 554 Mbits/sec
[ 4] 5.00-6.00    sec 69.5 MBytes 583 Mbits/sec
[ 4] 6.00-7.00    sec 64.2 MBytes 537 Mbits/sec
[ 4] 7.00-8.00    sec 66.2 MBytes 557 Mbits/sec
[ 4] 8.00-9.00    sec 70.1 MBytes 588 Mbits/sec
[ 4] 9.00-10.00   sec 69.1 MBytes 579 Mbits/sec
-----
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-10.00   sec 670 MBytes 561 Mbits/sec
[ 4] 0.00-10.00   sec 669 MBytes 561 Mbits/sec
iperf Done.

D:\iperf-3.1.3-win64>iperf3 -s
Server listening on 5201
Accepted connection from 192.168.0.101, port 41342
[ 5] local 192.168.0.102 port 5201 connected to 192.168.0.101 port 41348
[ ID] Interval      Transfer      Bandwidth
[ 5] 0.00-1.00    sec 87.2 MBytes 731 Mbits/sec
[ 5] 1.00-2.00    sec 91.0 MBytes 763 Mbits/sec
[ 5] 2.00-3.00    sec 90.9 MBytes 762 Mbits/sec
[ 5] 3.00-4.00    sec 90.7 MBytes 761 Mbits/sec
[ 5] 4.00-5.00    sec 90.8 MBytes 762 Mbits/sec
[ 5] 5.00-6.00    sec 90.7 MBytes 761 Mbits/sec
[ 5] 6.00-7.00    sec 90.8 MBytes 762 Mbits/sec
[ 5] 7.00-8.00    sec 90.9 MBytes 762 Mbits/sec
[ 5] 8.00-9.00    sec 90.8 MBytes 762 Mbits/sec
[ 5] 9.00-10.00   sec 90.8 MBytes 762 Mbits/sec
[ 5] 10.00-10.04  sec 3.63 MBytes 761 Mbits/sec
-----
[ ID] Interval      Transfer      Bandwidth
[ 5] 0.00-10.04   sec 0.00 Bytes 0.00 bits/sec
[ 5] 0.00-10.04   sec 908 MBytes 759 Mbits/sec
Server listening on 5201
```

```
root@image_7020:~# iperf3 -s
Server listening on 5201 (test #1)
Accepted connection from 192.168.0.102, port 5987
[ 5] local 192.168.0.101 port 5201 connected to 192.168.0.102 port 5988
[ ID] Interval      Transfer      Bitrate
[ 5] 0.00-1.00    sec 60.6 MBytes 509 Mbits/sec
[ 5] 1.00-2.00    sec 67.6 MBytes 567 Mbits/sec
[ 5] 2.00-3.00    sec 69.7 MBytes 585 Mbits/sec
[ 5] 3.00-4.00    sec 66.0 MBytes 553 Mbits/sec
[ 5] 4.00-5.00    sec 66.1 MBytes 554 Mbits/sec
[ 5] 5.00-6.00    sec 69.4 MBytes 583 Mbits/sec
[ 5] 6.00-7.00    sec 64.4 MBytes 540 Mbits/sec
[ 5] 7.00-8.00    sec 66.1 MBytes 554 Mbits/sec
[ 5] 8.00-9.00    sec 70.1 MBytes 588 Mbits/sec
[ 5] 9.00-10.00   sec 69.1 MBytes 578 Mbits/sec
[ 5] 10.00-10.01  sec 163 KBytes 365 Mbits/sec
-----
[ ID] Interval      Transfer      Bitrate
[ 5] 0.00-10.01  sec 669 MBytes 561 Mbits/sec
receiver
Server listening on 5201 (test #2)
^Ciperf3: interrupt - the server has terminated
root@image_7020:~# iperf3 -c 192.168.0.102
Connecting to host 192.168.0.102, port 5201
[ 5] local 192.168.0.101 port 41348 connected to 192.168.0.102 port 5201
[ ID] Interval      Transfer      Bitrate    Retr  Cwnd
[ 5] 0.00-1.01    sec 92.0 MBytes 762 Mbits/sec 1    421 KBytes
[ 5] 1.01-2.00    sec 90.0 MBytes 763 Mbits/sec 0    421 KBytes
[ 5] 2.00-3.01    sec 91.2 MBytes 762 Mbits/sec 0    421 KBytes
[ 5] 3.01-4.01    sec 91.2 MBytes 761 Mbits/sec 0    421 KBytes
[ 5] 4.01-5.00    sec 90.0 MBytes 762 Mbits/sec 0    421 KBytes
[ 5] 5.00-6.01    sec 91.2 MBytes 761 Mbits/sec 0    421 KBytes
[ 5] 6.01-7.01    sec 91.2 MBytes 762 Mbits/sec 0    421 KBytes
[ 5] 7.01-8.00    sec 90.0 MBytes 762 Mbits/sec 0    421 KBytes
[ 5] 8.00-9.01    sec 91.2 MBytes 762 Mbits/sec 0    421 KBytes
[ 5] 9.01-10.00   sec 90.0 MBytes 762 Mbits/sec 0    421 KBytes
-----
[ ID] Interval      Transfer      Bitrate    Retr
[ 5] 0.00-10.00   sec 908 MBytes 762 Mbits/sec 1
sender
[ 5] 0.00-10.00   sec 908 MBytes 762 Mbits/sec
receiver
iperf Done.
root@image_7020:~#
```

3.2 RS485 测试

开发板有两个 RS485 串口，将其并联连接。

dmesg |grep tty

可以看到两个 RS485 为 ttyS1、ttyS2。

```

root@image_7020:~# dmesg |grep tty
Kernel command line: console=ttyPS0,115200 earlycon root=/dev/mmcblk1p1 rw rootwait
e0000000.serial: ttyPS0 at MMIO 0xe0000000 (irq = 26, base_baud = 6249999) is a xuartps
printk: console [ttyPS0] enabled
43c10000.serial: ttyS2 at MMIO 0x43c10000 (irq = 40, base_baud = 6250000) is a 16550A
43c20000.serial: ttyS1 at MMIO 0x43c21000 (irq = 41, base_baud = 6250000) is a 16550A
root@image_7020:~#

```

因串口已被出厂测试程序占用，请使用出场程序进行测试。

3.3 CAN 测试

开发板有两个 CAN 接口，使用 `ifconfig -a` 命令可以看到开发板上有两路 can，即 can0 和 can1：

```

root@image_7020:~# ifconfig -a
can0: flags=193<UP, RUNNING, NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 1 bytes 5 (5.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1 bytes 5 (5.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 44

can1: flags=193<UP, RUNNING, NOARP> mtu 16
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 10 (UNSPEC)
    RX packets 1 bytes 5 (5.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1 bytes 5 (5.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 45

eth0: flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500
    inet 192.168.1.100 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::20a:35ff:fe00:1e53 prefixlen 64 scopeid 0x20<link>
    ether 00:0a:35:00:1e:53 txqueuelen 1000 (Ethernet)
    RX packets 30 bytes 2937 (2.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 33 bytes 5485 (5.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 46 base 0xb000

eth1: flags=67<UP, BROADCAST, RUNNING> mtu 1500
    inet 192.168.0.101 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::20a:35ff:fe00:122 prefixlen 64 scopeid 0x20<link>
    ether 00:0a:35:00:01:22 txqueuelen 1000 (Ethernet)
    RX packets 139 bytes 15278 (14.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 76 bytes 8865 (8.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP, LOOPBACK, RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 194 bytes 14156 (13.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 194 bytes 14156 (13.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sit0: flags=128<NOARP> mtu 1480
    unspec 00-00-00-00-31-00-00-4C-00-00-00-00-00-00-00-00 txqueuelen 1000 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@image_7020:~#

```

将 CAN1 和 CAN2 短接，进行回环测试，使用系统内置的脚本测试 CAN：

`./can_test.sh`

```

root@image_7020:~# ./can_test.sh
xilinx_can e0008000.can can0: bitrate error 0.0%
IPv6: ADDRCONF(NETDEV_CHANGE): can0: link becomes ready
xilinx_can e0009000.can can1: bitrate error 0.0%
IPv6: ADDRCONF(NETDEV_CHANGE): can1: link becomes ready
CAN0 is OK.
CAN1 is OK.
root@image_7020:~#

```

3.5 FLASH 测试

查看磁盘挂载情况

df -hT

```
root@image_7020:~# df -hT
Filesystem      Type      Size      Used Available Use% Mounted on
/dev/root        ext4       7.1G      536.7M    6.2G      8% /
devtmpfs        devtmpfs   493.6M     4.0K    493.6M    0% /dev
tmpfs           tmpfs      502.1M    408.0K    501.7M    0% /run
tmpfs           tmpfs      502.1M     56.0K    502.0M    0% /var/volatile
/dev/mmcblk0p2   ext4       3.0G      520.5M    2.4G     18% /media/sd-mmcblk0p2
/dev/mmcblk0p1   vfat       476.7M     8.1M    468.6M    2% /run/media/FAT-mmcblk0p1
root@image_7020:~#
```

EMMC 写速率测试:

time dd if=/dev/zero of=/test bs=1M count=1024

写入 1GB 的数据, 用时 40 秒, 速率约为 $1024/40=25$ MB/sec。

```
root@image_7020:~# time dd if=/dev/zero of=/test bs=1M count=1024
1024+0 records in
1024+0 records out

real    0m40.133s
user    0m0.012s
sys     0m12.461s
root@image_7020:~#
```

EMMC 读速率测试:

time dd if=/test of=/dev/zero bs=1M count=1024

读出 1GB 的数据, 用时 44 秒, 速率约为 $1024/44=23$ MB/sec。

```
root@image_7020:~# time dd if=/test of=/dev/zero bs=1M count=1024
1024+0 records in
1024+0 records out

real    0m44.677s
user    0m0.041s
sys     0m4.181s
root@image_7020:~#
```

SD 卡写速率测试:

time dd if=/dev/zero of=/media/sd-mmcblk1p1/test bs=1M count=1024

写入 1GB 的数据, 用时 43 秒, 速率约为 $1024/43=23$ MB/sec。

```
root@image_7020:~# time dd if=/dev/zero of=/media/sd-mmcblk1p1/test bs=1M count=1024
1024+0 records in
1024+0 records out

real    0m43.192s
user    0m0.000s
sys     0m12.202s
root@image_7020:~#
```

SD 卡读速率测试:

time dd if=/media/sd-mmcblk1p1/test of=/dev/zero bs=1M count=1024

读出 1GB 的数据, 用时 44 秒, 速率约为 $1024/44=23$ MB/sec。

```
root@image_7020:~# time dd if=/media/sd-mmcblk1p1/test of=/dev/zero bs=1M count=1024
1024+0 records in
1024+0 records out

real    0m44.619s
user    0m0.010s
sys     0m4.193s
root@image_7020:~#
```

3.6 QSPI 测试

查看 QSPI 的分区信息:

```

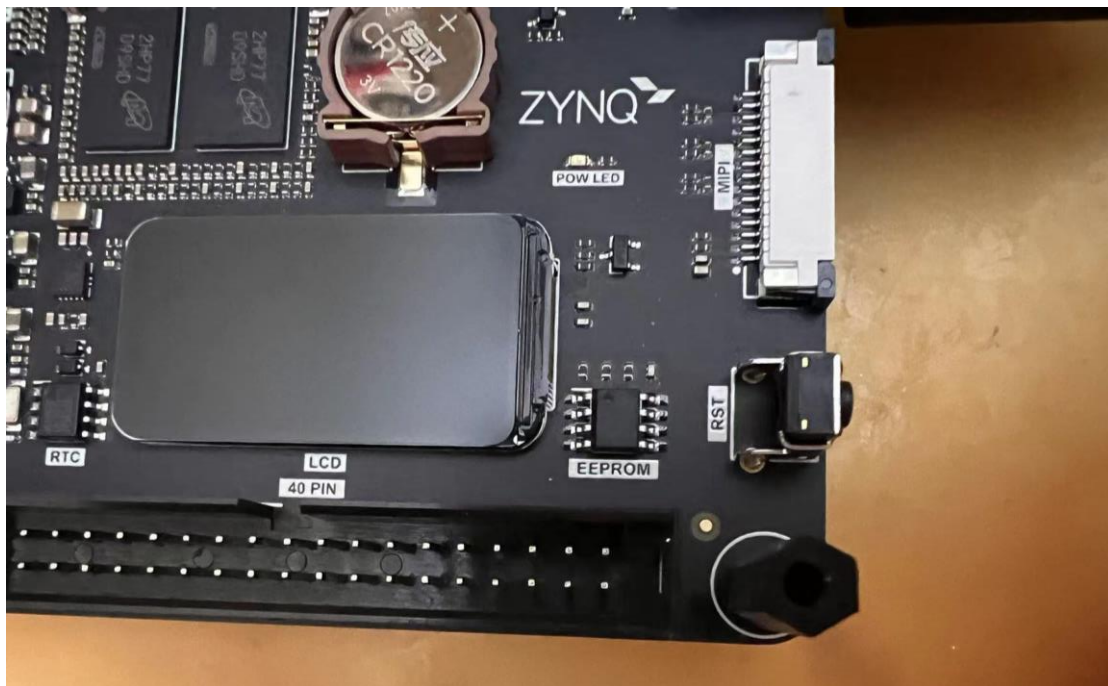
root@image_7020:~# cat /proc/mtd
dev:   size  erasesize  name
mtd0: 009b0000 00010000 "qspi-boot"
mtd1: 00010000 00010000 "qspi-bootenv"
mtd2: 00010000 00010000 "qspi-bootscr"
mtd3: 01500000 00010000 "qspi-kernel"
mtd4: 00130000 00010000 "space"
root@image_7020:~#

```

因其内部有启动文件，暂不进行读写测试。

3.10 EEPROM 测试

底板上搭载了一片 EEPROM，型号为 **AT24C256C-SSHL-T**，容量为：256Kbit，地址为 0x50，通过 I2C0 总线连接到 PS 端进行通信。



读取 EEPROM：

```
cat /sys/bus/i2c/devices/0-0050/eeprom
```

存储的日期为出厂测试日期，

```

root@image_7020:~# cat /sys/bus/i2c/devices/0-0050/eeprom
2024-09-17 09:13:11

```

写入 EEPROM：

```
echo "AABBCCDDEEFF" >/sys/bus/i2c/devices/0-0050/eeprom
```

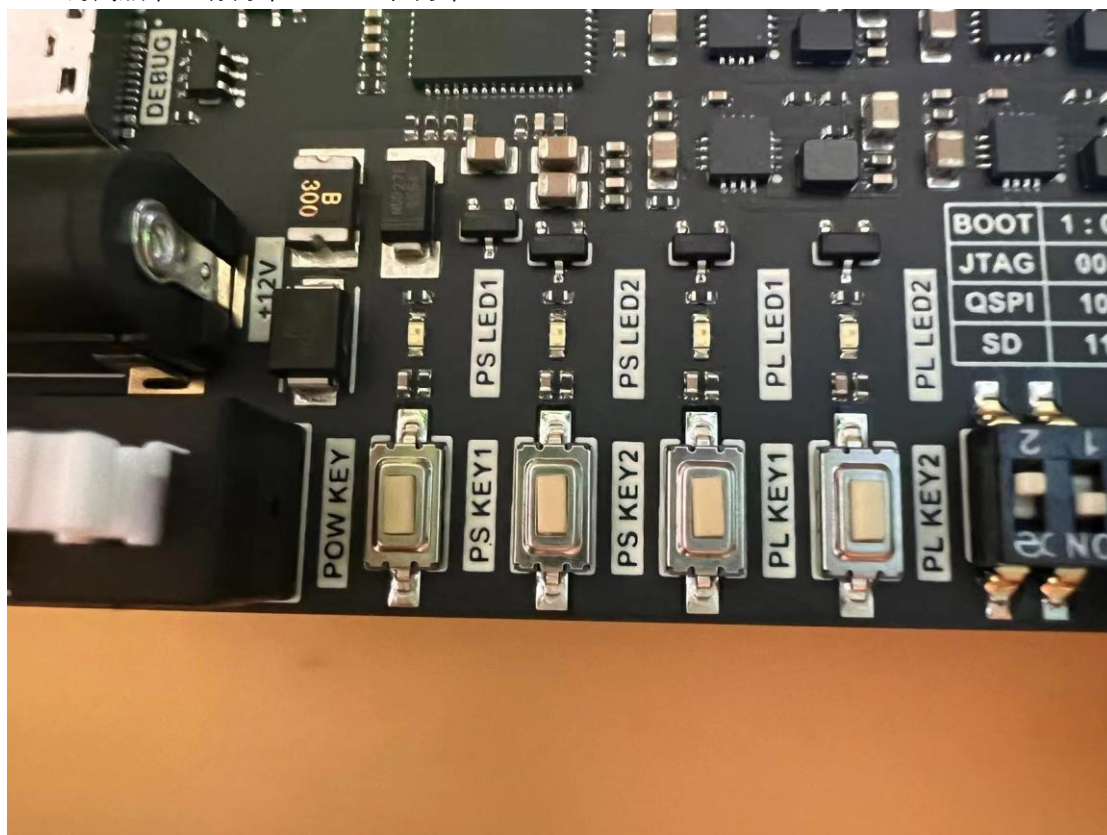
```

root@image_7020:~# echo "AABBCCDDEEFF" >/sys/bus/i2c/devices/0-0050/eeprom
root@image_7020:~# cat /sys/bus/i2c/devices/0-0050/eeprom
AABBCCDDEEFF
:13:11

```


3.7 LED 测试

测试版卡上有两个 PS LED 和两个 PL LED。



查看 leds 设备：

ls /sys/class/leds/

```
root@image_7020:~# ls /sys/class/leds/
input1::capslock  input7::capslock  input7::numlock  mmc1::  pl_prsnt
input1::numlock   input7::compose   input7::scrolllock  pl_led1  ps_led1
input1::scrolllock input7::kana       mmc0::  pl_led2  ps_led2
root@image_7020:~#
```

以 PL LED1 为例，

点亮 PL_LED1：

echo 1 > /sys/class/leds/pl_led1/brightness

关闭 PL_LED1：

echo 0 > /sys/class/leds/pl_led1/brightness

设置 PL_LED1 为心跳模式：

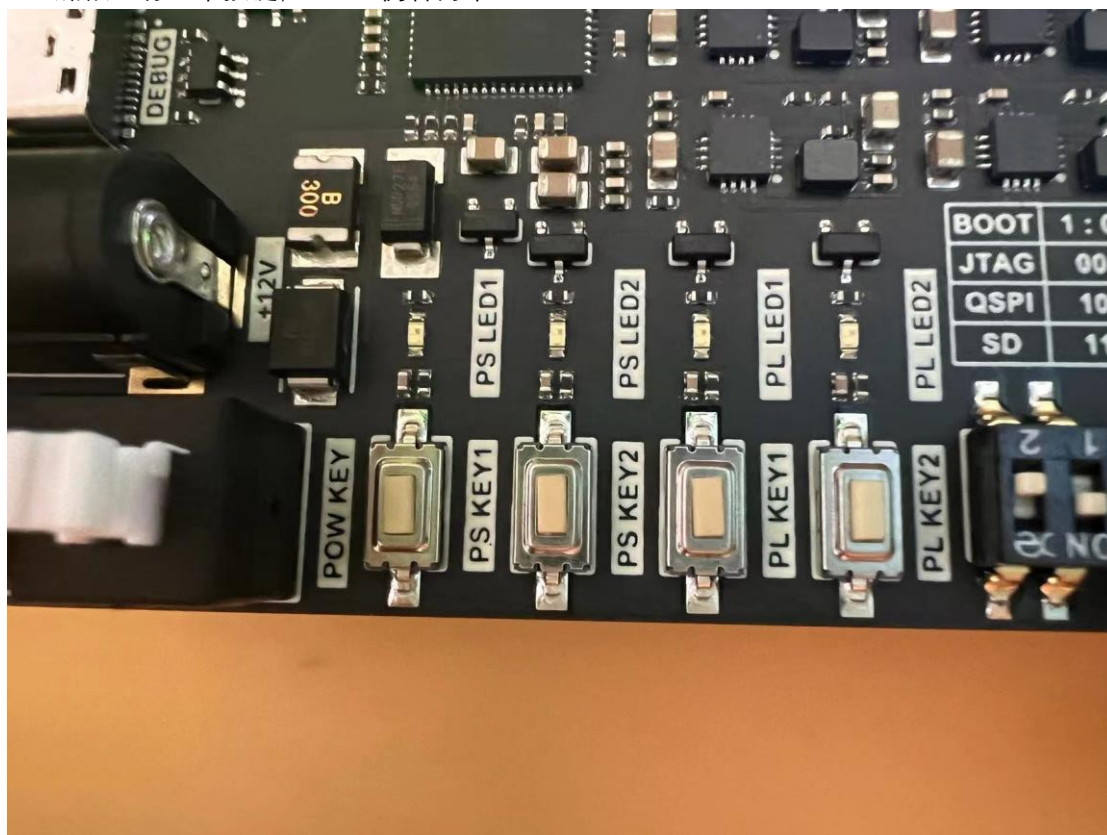
echo heartbeat > /sys/class/leds/pl_led1/trigger

设置 PL_LED1 为 timer 模式：

echo timer > /sys/class/leds/pl_led1/trigger

3.8 按键测试

底板上四个按键，PS PL 侧各两个。



使用 hexdump 命令捕获按键事件：

hexdump /dev/input/event0

有按键按下时则会在终端打印二进制文件，使用 ctrl+c 停止捕获。

```

root@image_7020:~# hexdump /dev/input/event0
00000000 e111 66e8 3cae 0003 0001 0067 0001 0000
00000010 e111 66e8 3cae 0003 0000 0000 0000 0000
00000020 e111 66e8 b205 0004 0001 0067 0000 0000
00000030 e111 66e8 b205 0004 0000 0000 0000 0000
00000040 e111 66e8 f8fd 0009 0001 006c 0001 0000
00000050 e111 66e8 f8fd 0009 0000 0000 0000 0000
00000060 e111 66e8 eb8b 000b 0001 006c 0000 0000
00000070 e111 66e8 eb8b 000b 0000 0000 0000 0000
00000080 e112 66e8 73e6 0002 0001 0069 0001 0000
00000090 e112 66e8 73e6 0002 0000 0000 0000 0000
000000a0 e112 66e8 aff2 0004 0001 0069 0000 0000
000000b0 e112 66e8 aff2 0004 0000 0000 0000 0000
000000c0 e112 66e8 59d4 000c 0001 006a 0001 0000
000000d0 e112 66e8 59d4 000c 0000 0000 0000 0000
000000e0 e112 66e8 f0c7 000d 0001 006a 0000 0000
000000f0 e112 66e8 f0c7 000d 0000 0000 0000 0000

```

3.9 RTC 测试

查看系统时钟：

```
data
```

查看硬件时钟：

```
hwclock
```

```
root@image_7020:~# date
Tue Sep 17 09:54:16 CST 2024
root@image_7020:~# hwclock
Tue Sep 17 01:54:18 2024 0.000000 seconds
root@image_7020:~#
```

设置系统时钟为 2024 年 10 月 1 号 08 点 08 分 08 秒：

```
date -s "2024-10-1 08:08:08"
```

将系统时钟写入 RTC 时钟：

```
hwclock -w
```

```
root@image_7020:~# date -s "2024-10-1 08:08:08"
Tue Oct 1 08:08:08 CST 2024
root@image_7020:~# hwclock -w
root@image_7020:~# date
Tue Oct 1 08:08:32 CST 2024
root@image_7020:~# hwclock
Tue Oct 1 08:08:34 2024 0.000000 seconds
root@image_7020:~#
```

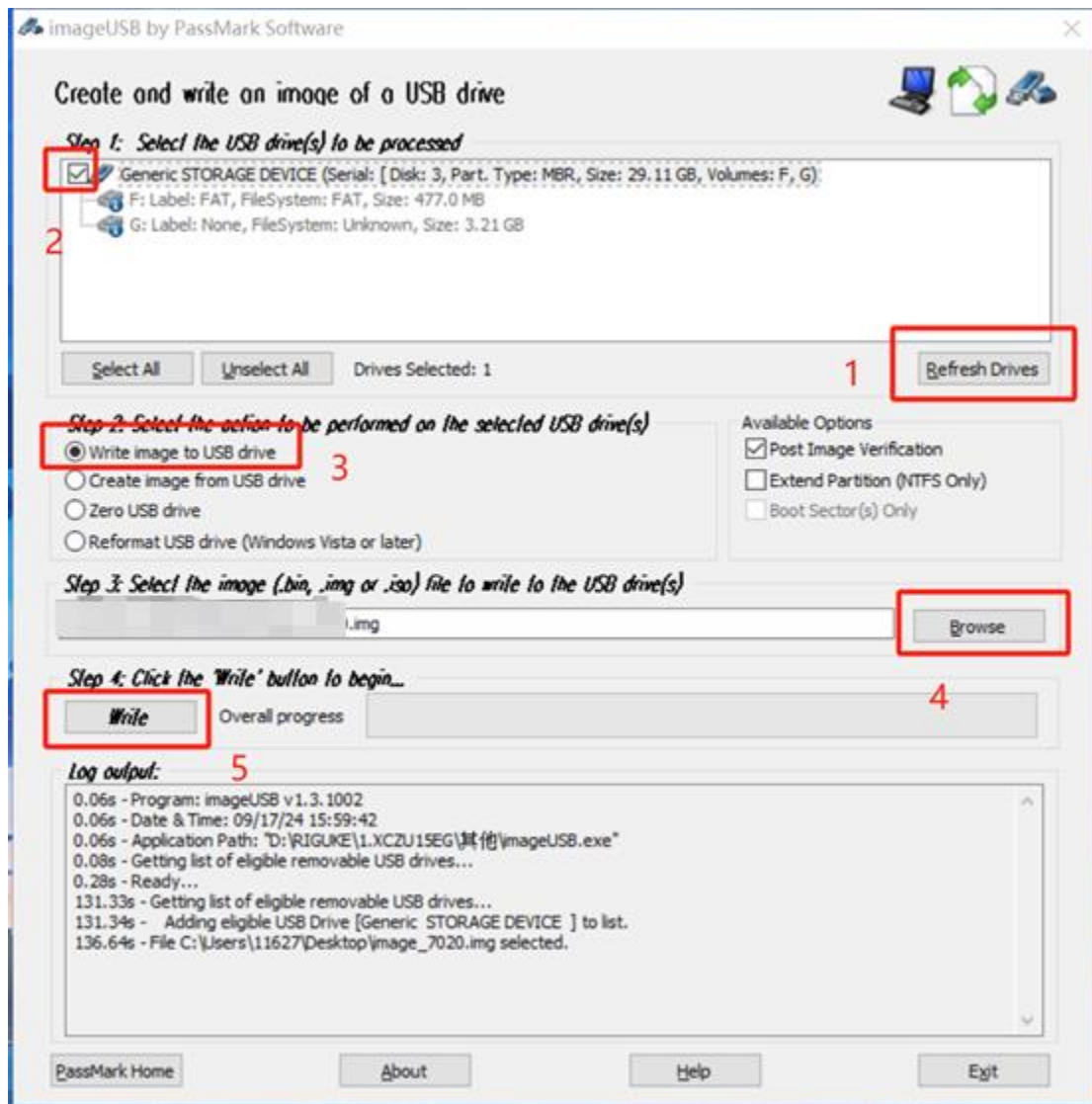
使用 NTP 时间同步（需主机安装 NTP 服务）

```
ntpdate -u 192.168.1.1
```

```
root@image_7020:~# ntpdate -u 192.168.1.1
17 Sep 09:55:49 ntpdate[687]: step time server 192.168.1.1 offset -1203209.735631 sec
root@image_7020:~#
```

三 镜像烧录进 SD 卡

1. 使用 SD 卡和读卡器连接至电脑，并格式化为 **FAT32** 格式；
2. 下载并打开网盘中软件安装包内的 imageUSB.exe 工具；
3. 按如下步骤烧录 SD 卡镜像：



也可以在 linux 下使用 dd 命令烧录进 SD 卡（一般的 SD 卡对应 sdb 设备，客户可自行查看）：

```
dd if=image_7020.img of=/dev/sdb
```


四 镜像烧录进 QSPI 卡

1. 使用《三 镜像烧录进 SD 卡》章节中烧录完成的 SD 卡启动 linux
2. 启动完成后，运行以下命令

`./burn_qspi.sh`

```

root@image_7020:~# ./burn_qspi.sh
#####
##### RK-ZYNQ7020-F Flash QSPI #####
#####

Erase partition /dev/mtd0
Burn BOOT.BIN to QSPI
Erase partition /dev/mtd1
Erase partition /dev/mtd2
Burn boot.scr to QSPI
Erase partition /dev/mtd3
Burn image.ub to QSPI
EXT4-fs (mmcblk1p1): unmounting filesystem.
4+0 records in
4+0 records out
Create EXT4 partition for eMMC
mmcblk1: p1
mmcblk1: p1
EXT4-fs (mmcblk1p1): mounted filesystem with ordered data mode. Quota mode: disabled.
EXT4-fs (mmcblk1p1): unmounting filesystem.
Format EXT4 partition
mke2fs 1.46.5 (30-Dec-2021)
Mount the EXT4 partition
EXT4-fs (mmcblk1p1): mounted filesystem with ordered data mode. Quota mode: disabled.
Copy roots to eMMC
sync .....
Unmount partition
EXT4-fs (mmcblk1p1): unmounting filesystem.
##### Complete #####
root@image_7020:~#

```

3. 约五分钟完成，板上的四个 LED 全亮，关闭电源，拨码开关拨至 QSPI 启动，即可使用 QSPI 启动出场镜像。

